DEVELOPMENT OF A PREDICTIVE MAINTENANCE MODEL FOR COILED EVAPORATORS IN SPLIT-TYPE AIR CONDITIONING SYSTEMS INSTALLED IN SBM AVR FACILITIES, THE COLLEGE OF ENGINEERING, FABER HALL OF XAVIER UNIVERSITY CAGAYAN DE ORO CITY

John Ronald Pacaldo

Collin Brandon Asio

Simon France Sulibio

Joel P. Rudinas Jr.

ACE 17.5 - EF

September 2025

CERTIFICATE OF ORIGINALITY

This is to certify that we assume full responsibility over the work entitled “DEVELOPMENT OF A PREDICTIVE MAINTENANCE MODEL FOR COILED EVAPORATORS IN SPLIT-TYPE AIR CONDITIONING SYSTEMS INSTALLED IN THE COLLEGE OF ENGINEERING BUILDING, FABER HALL, AND SBM AVR AT XAVIER UNIVERSITY – ATENEO DE CAGAYAN” submitted as a requirement for ACE 17.5 , Research and Engineering Design Methods for ME at Xavier University – Ateneo de Cagayan, that the work is our own; that this is original except as specified in the acknowledgements, footnotes, or in the references and that this has never been submitted to this or any other school for a degree or other requirements.

COLLIN BRANDON ASIO

JOHN RONALD PACALDO

SIMON FRANCE SULIBIO

CERTIFICATION

This is to certify that Chapter 1 of the research proposal entitled: “DEVELOPMENT OF A PREDICTIVE MAINTENANCE MODEL FOR COILED EVAPORATORS IN SPLIT-TYPE AIR CONDITIONING SYSTEMS INSTALLED IN THE COLLEGE OF ENGINEERING BUILDING, FABER HALL, AND SBM AVR AT XAVIER UNIVERSITY – ATENEO DE CAGAYAN” has been reviewed and duly approved by the undersigned Research Adviser. The submitted chapter has been evaluated for clarity, feasibility, and alignment with research standards.

Furthermore, I, **Engr. Joel P. Rudinas Jr.**, affirm my acceptance to serve as the Primary Research Adviser of the group and to provide continuous guidance and supervision throughout the conduct of their research.

This certification is issued to confirm that the proponents may proceed with the succeeding chapters of their research. Issued this 21 day of October, 2025 at Xavier University – Ateneo de Cagayan, Cagayan de Oro City.

Checked and Approved by:

Engr. Joel P. Rudinas Jr.

Research Adviser

Endorsed by:

Dr. Rogelio C. Golez

Research Professor

Noted by:

Dr. Elmer B. Dollera

Chairman, Mechanical Engineering Department

TABLE OF CONTENTS

CERTIFICATE OF ORIGINALITY 1

TABLE OF CONTENTS 4

CHAPTER I: INTRODUCTION 6

CHAPTER II: REVIEW OF RELATED LITERATURE 15

2.1 AI-Driven Predictive Maintenance in HVAC Systems: Strategies for Improving Efficiency and Reducing System Downtime [1] 16

2.2 Predictive Maintenance of Air Conditioning Systems Using Supervised Machine Learning [2] 18

2.3 Predictive Maintenance Strategies for HVAC Systems: Leveraging MPC, Dynamic Energy Performance Analysis, and ML Classification Models [3] 20

2.4 Fault detection for air conditioning system using machine learning [4] 21

2.5 Predictive Maintenance in Building Facilities: A Machine Learning Approach. Sensors [5]23

2.6 Machine Learning Algorithms for Predictive Maintenance in HVAC Systems [6] 25

2.7 Research on Fault Diagnosis Strategy of Air-Conditioning Systems Based on DPCA and Machine Learning [7] 27

2.8 Predictive maintenance magnetic sensor using random forest method [8] 29

2.9 Predictive maintenance of electromechanical systems based on enhanced generative adversarial neural network with convolutional neural network. [9] 29

2.10 Machine Learning: Algorithms, Real-World Applications and Research Directions [10]30

CHAPTER III: METHODOLOGY 37

3.1 DATA COLLECTION 38

3.1.1 RESEARCH DESIGN OVERVIEW 38

3.1.2 SOURCES OF DATA....38
3.1.3 SAMPLING....40
3.1.4 HARDWARE USED....40
3.1.5 DATA GATHERING PROCEDURE....42
3.1.6 VALIDITY AND RELIABILITY MEASURES....42
3.1.7 DATA MANAGEMENT AND STORAGE....52
3.2 DATA PREPROCESSING....56
3.2.1 DATA CLEANING....57
3.2.2 DATA SPLITTING....57
3.2.3 DATA TRANSFORMATION....59
3.2.4 DATA REDUCTION....61
3.3 MODEL DEVELOPMENT....64
3.4MODEL EVALUATION....90
3.5MODEL DEPLOYMENT....94
8.1. Deployment Architecture and Technology Selection....95
3.6GENERAL PROJECT WORKFLOW....98
CHAPTER IV: RESULTS AND DISCUSSION....101
4.1 SENSORS VALIDITY EXPERIMENT....102
4.1.1 CALIBRATION OF BME 280....102
4.1.2 CALIBRATION OF DS18B20....106
REFERENCES....111
APPENDIX....113
A.1 BME 280 EXPERIMENTAL CODE....114

A.2 BME 280 VALIDITY EXPERIMENT TABLE 116

A.3 BME 280 VALIDITY CALIBRATED CODE 124

A.4 DS18B20 PROBE A TABLE.....126

A.5 DS18B20 PROBE B TABLE.....135

A.6 DS18B20 PROBE C TABLE 143

A.7 DS18B20 EXAMPLE EXPERIMENTAL CODE 152

CHAPTER I: INTRODUCTION

1.1 Background of the Study

Air conditioning has mostly become a necessity in this modern world to achieve thermal comfort indoors, particularly in hot and humid climates. It involves the process of cooling and dehumidifying air within the confines of a building—be it residential, commercial, or industrial. However, air conditioning systems account for a considerable portion of a building's energy consumption, especially in tropical regions. The split-type air conditioner is widely used due to its energy efficiency, compact design, and ease of installation (Chaktranond & Doungsong, 2010). The efficiency and reliability of the evaporator coil directly influence the cooling performance, energy consumption, and overall lifespan of the unit.

However, evaporator coils in split-type air conditioners are susceptible to performance degradation from dust accumulation, corrosion, microbial growth, refrigerant issues, and poor maintenance. These factors lead to reduced heat transfer, increased energy use, and potential system failure. Experimental tests show that fouling just 30% of the surface of an evaporator can cause the Energy Efficiency Ratio (EER) to drop by 13.5%, increase energy consumption by 6.4%, and reduce cooling load by 19.1% (Niknami et al., 2024). Traditionally, maintenance practices have been preventive or scheduled, often leading to unnecessary service interruptions or unoptimized maintenance schedules.

This calls for the implementation of **Predictive Maintenance (PdM)**, which utilizes Artificial Intelligence (AI), the Internet of Things (IoT), and real-time monitoring to make HVAC systems more intelligent and efficient. PdM enables early fault detection, optimized servicing, enhanced reliability, and improved energy efficiency (Tejani, 2024). The development of such a model for coiled

evaporators in split-type systems is essential for cost reduction, user comfort, and sustainable energy use.

Moreover, this initiative aligns with the objectives of the **Republic Act No. 11285, also known as the *Energy Efficiency and Conservation Act*,** which promotes energy efficiency and the judicious conservation of energy in the Philippines. The law mandates the adoption of energy-efficient technologies and practices in both public and private sectors to reduce overall energy consumption and environmental impact. Developing a predictive maintenance model contributes to this national goal by ensuring that air conditioning systems operate at optimal efficiency, minimizing waste, and supporting the country's sustainable development efforts.

Traditional maintenance practices for split-type aircons in Xavier University are primarily limited to periodic cleaning conducted by third-party service providers every three to four months. Based on the interview with *the personnel in charge from the PPO*, maintenance activities are largely embedded within the cleaning process, following external standards set by contracted companies, rather than being driven by continuous condition monitoring. While this approach assumes that cleaning equates to inspection and correction as the personnel has said, instances of system damage—such as leaks and vibration-related issues—have been observed to occur within as little as two months after servicing. This indicates a gap between scheduled maintenance intervals and the actual operational condition of the units. In addition, the reliance on multiple third-party companies and different technicians increases maintenance costs and introduces variability in service quality, despite the persistence of faults between cleaning cycles. At present, there is no localized, data-driven mechanism that provides early warnings or continuous assessment of system health, resulting in undetected issues that may escalate into costly repairs. This gap highlights the need for a predictive maintenance approach tailored to split-type air-conditioning systems in the PPO, where machine learning–based models can support early fault detection, reduce unplanned damage, and improve cost efficiency while complementing existing maintenance practices rather than relying solely on periodic cleaning schedules.

As HVAC systems continue to grow more complex, traditional maintenance approaches—such as reactive and preventive methods—have proven less effective. Reactive maintenance only addresses

issues after a failure occurs, while preventive maintenance may result in unnecessary servicing or overlooked needs. Predictive Maintenance (PdM), powered by AI and Machine Learning (ML), provides a proactive solution by anticipating potential failures and reducing downtime. Through real-time data analysis from sensors and IoT devices, AI technologies such as machine learning and neural networks can identify patterns, predict breakdowns, and optimize system performance. This integration of AI in HVAC maintenance not only enhances reliability and conserves energy but also upholds the principles of energy efficiency and sustainability outlined in **RA 11285**. This study aims to answer the following questions:

1. How effective are the chosen data preprocessing and statistical feature selection techniques in identifying the relevant input data from IoT sensors and system logs?
2. Which of the tested predictive models demonstrates the highest reliability in classifying the system as either normal or abnormal?
3. How can the proposed framework be designed to successfully integrate real-time data collection and model analysis into a cloud-based or self-hosted proactive maintenance workflow?

1.2 Main Objective

The main objective of this study is to design and develop a predictive maintenance model for coiled evaporators in split-type air conditioning systems using artificial intelligence techniques. This research aims to analyze critical operational parameters, including air temperature, ice build-up, compressor current, and supply voltage, to accurately anticipate potential failures by classifying whether the given system is undergoing normal or abnormal conditions and optimize maintenance schedules. The model will focus on AI-based analysis as the core mechanism for prediction, while IoT-enabled data collection may serve as a supplementary approach to enhance real-time monitoring capabilities. By achieving this objective, the study intends to minimize system downtime, reduce maintenance costs, and improve the overall reliability and efficiency of split-type air conditioning systems.

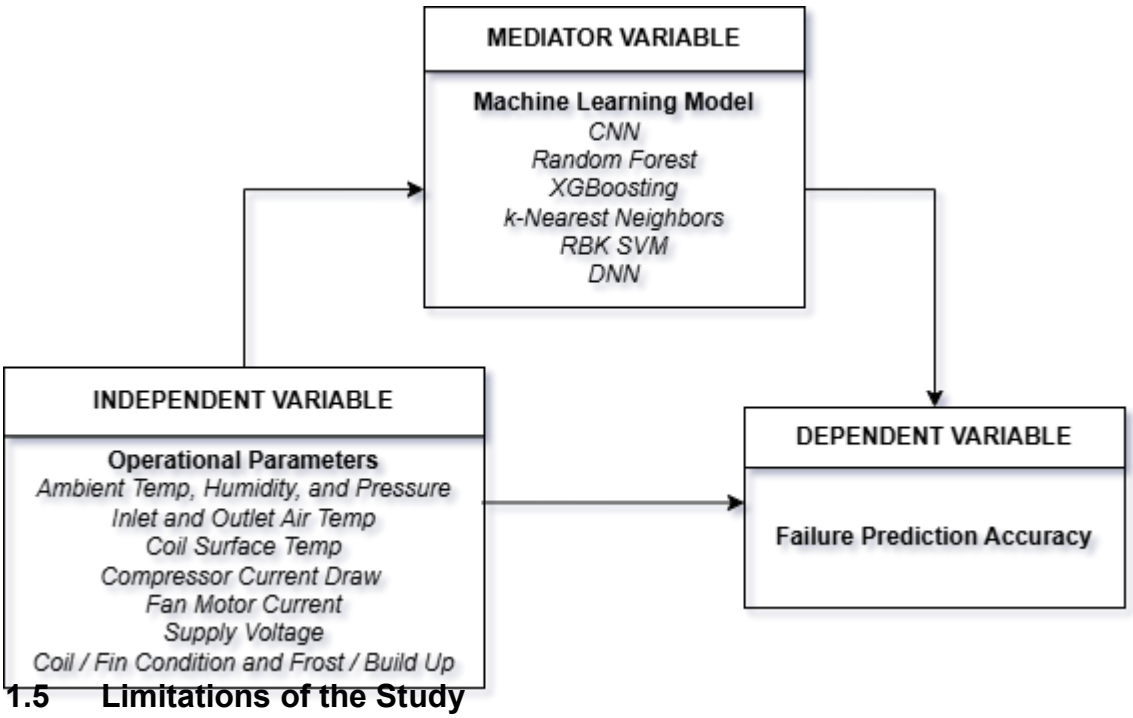
1.3 Specific Objectives

To accomplish the main objective, this study sets out the following specific objectives:

- To establish a data acquisition and preprocessing pipeline, and to perform feature selection/extraction using statistical analysis to ensure the quality and relevance of input data gathered from various sources such as IoT sensors and system logs.
- To select and validate top-performing predictive model by conducting comparison tests between Hybrid Model: Random Forest and CNN, XGBoost, k-NN, RBK SVM and DNN using KPIs such as Precision (accuracy), sensitivity, and F1 scores.
- To propose a framework that incorporates proactive maintenance or maintenance 4.0 through the use of cloud-based or self-hosted workflow which incorporates real-time data collection, model analysis, and system monitoring.

1.4 Conceptual Framework

The conceptual framework of this study illustrates the interaction between the independent, mediating, and dependent variables within the context of predictive maintenance for coiled evaporators in split-type air conditioning systems. The **independent variables** consist of critical operational parameters—namely air temperature, noise, ice build-up, and refrigerant leaks—which serve as indicators of the system's performance and condition. These variables do not exert a direct influence on the maintenance outcome; rather, their effect is mediated through the **machine learning model**, which functions as the **mediating variable**. This model employs artificial intelligence techniques to analyze the collected data and identify patterns or anomalies that may indicate potential failures. Additionally, the integration of Internet of Things (IoT) technology facilitates real-time data acquisition, enhancing the accuracy and timeliness of the analysis. The **dependent variable** represents the predictive maintenance outcome, which pertains to the model's capability to accurately forecast failures and support proactive maintenance strategies. This framework underscores the pivotal role of the AI-driven model as the mechanism that translates raw sensor data into actionable insights, thereby improving system reliability and maintenance efficiency.



1.5 Limitations of the Study

The researchers recognize that this study possesses multiple limitations that should be taken into account when analyzing its results. The research offers significant insights into the formulation of a predictive maintenance model for coiled evaporators in split-type air conditioning systems; however, the limitations of scope and resources inevitably restricted certain facets of the study.

Data Collection Is Time-Consuming

The first limitation arises from the time needed to get accurate data. Coil fouling, which has a big effect on how well an evaporator works, usually happens over the course of several weeks or months. The study's short length meant that only short-term monitoring was possible. To simulate fouling, cloth or mesh was employed to obstruct airflow; however, this technique fails to accurately mimic the natural accumulation of dust, microbial proliferation, or corrosion. Research indicates that even partial fouling of evaporators can significantly diminish efficiency and cooling capacity (Niknami et al., 2024). This constraint diminishes the external validity of the model, as forecasts produced within a two- to three-week timeframe may not comprehensively reflect real-world, long-term application.

Consequently, subsequent research ought to prolong monitoring durations and incorporate authentic operational settings to acquire more representative datasets.

Limited Sensor Accuracy Due to Budget Constraints

Another limitation has to do with the tools used. The predictive maintenance model relies on sensor data; however, budget limitations necessitated the use of low-cost sensors with error margins of ±2–5%. These mistakes could add noise to the dataset, making predictions less reliable and increasing the chance of false alarms or missed problems. This study did not include professional research-grade sensors because they were too expensive. Prior research has underscored that sensor quality is essential for dependable predictive maintenance results (Tejani, 2024). Future research may mitigate this limitation by employing calibration, fostering collaborations with laboratories, or incorporating hybrid systems that amalgamate cost-effective and high-precision devices.

Setup Cost and Integration Challenges

The researchers are also aware of the problems that come with setting up and integrating. For predictive maintenance to work, you often need to install and coordinate several sensors, which can be hard to do from a technical and financial point of view. In the context of this study, inadequate infrastructure constrained the intricacy of integration and data management. These limitations could have an impact on the model's ability to be replicated and its strength. According to industry reports, one of the main problems with predictive maintenance systems is that they require a lot of money to start up and can be hard to integrate (Banetti, 2023). Future research may implement a modular integration strategy, commencing with fundamental sensors and progressively expanding.

Modeling Complexity Versus Student Expertise

The researchers also recognize that the complexity of predictive modeling presents a limitation relative to their current level of expertise. For this study, machine learning models are limited to classifying the overall condition of the split-type air conditioning system and its evaporator coil as either normal or abnormal. Implementing a regression-based approach would require more advanced analytical skills and highly accurate sensor data, both of which are currently lacking. Without precise numerical measurements to quantify the system's condition, the researchers cannot reliably develop or validate a regression model, making classification the more feasible option for this project.

Notwithstanding these drawbacks, the study offers a useful starting point for comprehending predictive maintenance in coiled evaporators, the researchers stress. It shows that creating a predictive model with readily available resources is feasible and identifies areas for system reliability, cost reduction, and energy efficiency. The study makes a significant contribution to current research on more intelligent HVAC maintenance techniques by pointing out limitations and suggesting future directions.

1.6 Definition of Terms

- *Anomaly Detection* – A technique used in machine learning and data analysis to identify unusual patterns or deviations from normal system behavior. In this study, anomaly detection helps detect early signs of evaporator faults.
- *Artificial Intelligence (AI)* – A branch of computer science that enables machines to mimic human intelligence processes such as learning, reasoning, and problem-solving. AI in this research is used for predictive analysis of system parameters.
- *Coiled Evaporator* – A heat exchange component in an air conditioning system responsible for absorbing heat from indoor air, enabling the refrigerant inside the coil to evaporate and produce a cooling effect.

- *Convolutional Neural Network (CNN)* – A deep learning algorithm widely used for image recognition and analysis. In this study, CNN will be applied to analyze heat map images for detecting ice build-up on evaporator coils.
- *Dataset* – A structured collection of data points, including system parameters like air temperature, noise levels, refrigerant status, and ice formation, used to train and evaluate machine learning models in this research.
- *Feature Extraction* – The process of selecting and transforming relevant input variables (features) from raw data to be used in machine learning models for better predictive accuracy.
- *Heat Map* – A graphical representation where colors indicate different intensity levels of a variable, such as temperature or ice formation. This research uses heat maps for visualizing ice accumulation on evaporator coils.
- *Ice Build-Up* – The accumulation of ice on the evaporator coil, which restricts airflow and decreases cooling efficiency. Detecting and predicting ice build-up is critical for system reliability.
- *Internet of Things (IoT)* – A network of interconnected devices that communicate and share data through the internet. IoT-enabled sensors in this study may be used for real-time monitoring of temperature, noise, and refrigerant status.
- *Machine Learning (ML)* – A subset of AI that uses algorithms and statistical models to learn patterns from data and make predictions or decisions without explicit programming.
- *Model Accuracy* – A performance metric that measures how correctly a predictive model forecasts outcomes compared to actual results. In this study, accuracy indicates the reliability of the predictive maintenance model.
- *Neural Network (NN)* – A machine learning model inspired by the human brain, composed of layers of interconnected nodes (neurons) that process input data to learn complex patterns and make predictions.
- *Noise* – Unwanted or abnormal sound from the air conditioning system components. Noise levels can indicate mechanical wear or faults in the evaporator.

- *Overfitting* – A modeling error in machine learning where a model learns the training data too well, including its noise, resulting in poor performance on unseen data. Avoiding overfitting is important for building a reliable predictive maintenance model.
- *Predictive Maintenance* – A proactive maintenance approach that predicts potential failures before they occur by analyzing historical and real-time data, reducing unplanned downtime and costs.
- *Random Forest* – An ensemble machine learning algorithm that builds multiple decision trees and combines their outputs to improve accuracy. It is useful for handling large datasets with multiple parameters.
- *Refrigerant Leak* – The unintended escape of refrigerant gas from the system, which leads to reduced cooling efficiency and can damage system components if not addressed promptly.
- *Sensor Data* – Information collected by IoT-enabled sensors such as temperature readings, sound levels, and refrigerant status, which serves as input for machine learning algorithms.
- *Split-Type Air Conditioning System* – A cooling system consisting of two separate units: an indoor unit (with the evaporator coil) and an outdoor unit (with the condenser). Common in residential and small commercial applications.
- *Temperature (Air Temperature)* – The measure of thermal conditions around or inside the air conditioning system. Monitoring fluctuations in air temperature helps identify system inefficiencies or malfunctions.
- *Validation* – The process of testing a machine learning model with unseen data to ensure its accuracy and generalizability in real-world conditions.

CHAPTER II: REVIEW OF RELATED LITERATURE

2.1 AI-Driven Predictive Maintenance in HVAC Systems: Strategies for Improving Efficiency and Reducing System Downtime [1]

Tejani, A., 2024, ESP IJAST, Vol 2 Issue 3

Tejani, A. (2024). The study of AI-driven predictive maintenance in HVAC systems has gained major attention in recent years due to the growing demand for energy efficiency and system reliability. According to Ankitkumar Tejani (2024), traditional maintenance methods such as reactive and preventive maintenance often lead to unnecessary operational costs, system inefficiencies, and downtime. In contrast, predictive maintenance (PdM) employs artificial intelligence (AI) and machine learning (ML) algorithms to forecast potential equipment failures before they occur, thereby minimizing interruptions and improving overall system performance. Predictive maintenance revolves around concepts of *Condition-Based Monitoring (CBM)* and *Prognostics and Health Management (PHM)*. These frameworks depend on continuous data collection through sensors and Internet of Things (IoT) devices to monitor system health parameters such as temperature, vibration, humidity, and airflow. The data are then analyzed through machine learning models—such as Support Vector Machines (SVM), Random Forests, K-Means Clustering, and Q-Learning—to identify anomalies and predict *Remaining Useful Life (RUL)* of components. The reviewed studies primarily aim to: Improve the reliability and lifespan of HVAC systems through predictive maintenance, reduce energy consumption by identifying performance inefficiencies, minimize system downtime and repair costs by detecting faults before they occur, integrate AI and IoT technologies for real-time decision-making in HVAC maintenance. The study includes workflow in *Data Preprocessing, Model Training and Validation, Integration with HVAC Systems*. These workflows are in a form of flowcharts.

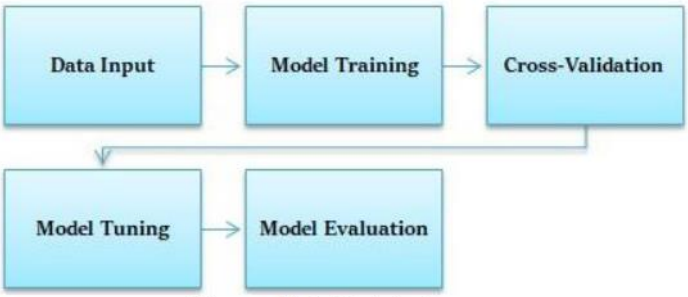
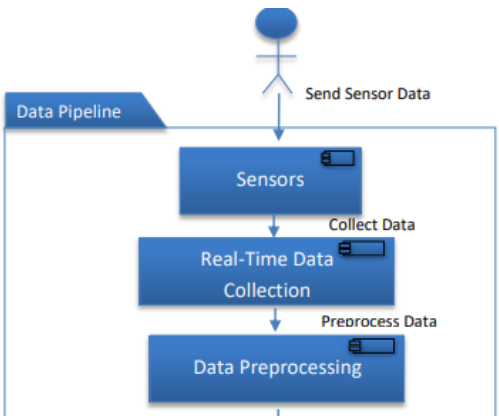


Figure 2: Model Training Process



The research presents a **Predictive Maintenance System Architecture**, which integrates sensor networks, real-time data pipelines, and AI predictive engines. This architecture (illustrated as Figure 3 in Tejani's work) emphasizes the interaction between data collection, analysis, and actionable decision-making—making it a clear representation of a closed-loop maintenance system. The existing literature, particularly the work of Ankitkumar Tejani (2024)**,** establishes a strong foundation for the integration of AI and predictive analytics in HVAC system maintenance. The combination of real-time monitoring, data-driven modeling, and automated alerts demonstrates how maintenance can shift from a reactive to a proactive paradigm. However, the unresolved issues in scalability, data integrity, and ethical governance open further opportunities for research in autonomous predictive maintenance systems, especially in industrial and large-scale applications.

2.2 Predictive Maintenance of Air Conditioning Systems Using Supervised Machine Learning [2]

Trivedi, S., Bhola, S., Taelgaonkar, S., Gaur, P., 2019, ISAP

Trivedi et al (2019). The study of predictive maintenance in air conditioning systems continues to evolve with the growing use of artificial intelligence and data-driven techniques. In their conference paper, Trivedi et al. (2019) proposed an innovative approach using supervised machine learning algorithms—specifically Decision Tree and Support Vector Machine (SVM)—to detect common air conditioning faults such as gas leakage and capacitor malfunction. The research highlights how integrating distributed sensing and real-time monitoring can significantly reduce energy losses and system downtime by identifying faults early. The paper introduces the concept of predictive maintenance (PdM) as a data-driven maintenance strategy that relies on continuous monitoring through sensors. This differs from traditional preventive or reactive maintenance by predicting when a fault will occur before it causes major system failure. The system uses a distributed sensor network composed of voltage and current transformers to measure electrical parameters such as real power, reactive power, apparent power, and power factor. These parameters serve as diagnostic indicators for the health of the air conditioning unit. The authors' main objectives were to: develop a predictive maintenance model capable of identifying the operational status and fault type in AC units: compare the accuracy of machine learning classifiers in fault detection and load identification. Create a real-time monitoring prototype that collects sensor data, processes it through a microcontroller, and trains algorithms to classify faults effectively. Evaluate performance using real experimental data and benchmark datasets (IAWE 2013).Through this setup, the study achieved a fault detection accuracy of 93.5% and load identification accuracy of 93.6%, with the Fine Decision Tree algorithm performing better than SVM classifiers.

The paper presents a clear **block diagram** of their predictive maintenance prototype. The system consists of:

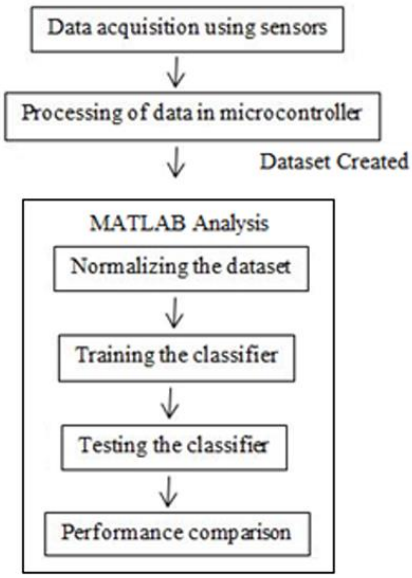


Figure 1. Block Diagram of the Proposed Model

The study by **Trivedi et al. (2019)** stands as an important contribution to the field of predictive maintenance for HVAC systems. By successfully combining hardware prototyping, distributed sensing, and supervised learning, it demonstrates how AI can transform fault detection into a proactive and data-driven process. The work not only validates the high performance of decision tree classifiers but also bridges the gap between theoretical approaches and real-world application. Despite certain data and scalability limitations, this research provides a strong foundation for developing intelligent monitoring systems capable of optimizing energy use and reducing maintenance costs in air conditioning networks.

2.3 Predictive Maintenance Strategies for HVAC Systems: Leveraging MPC, Dynamic Energy Performance Analysis, and ML Classification Models [3]

Singh, D., Arshad, M., Tyagi, B., & Kalia, G., 2023, IRE Journals, Vol. 7 Issue 4

Singh, Arshad, Tyagi, and Kalia (2023) an advanced predictive maintenance framework for Heating, Ventilation, and Air Conditioning (HVAC) systems by integrating Model Predictive Control (MPC), machine learning (ML) classification models, and dynamic energy performance benchmarking. Their

work represents one of the more comprehensive approaches to predictive maintenance, focusing not only on detecting equipment faults but also on optimizing system performance and reducing operational energy consumption. Predictive maintenance (PdM) aims to predict when equipment failure might occur so that maintenance can be scheduled just in time to prevent it. The authors emphasize that HVAC systems are highly nonlinear and composed of multiple interacting subsystems such as fans, dampers, filters, and coils. As such, modeling their performance using traditional physics-based forward modeling is computationally expensive. The paper instead uses data-driven modeling through Random Forest (RF) and Logistic Regression (LR) algorithms trained on the Semiconductor Manufacturing Process (SECOM) dataset, simulating HVAC behavior under different operational and fault conditions. The research by Singh et al. had several key objectives: To develop a machine learning–based predictive maintenance model capable of classifying faults in HVAC systems. To introduce a dynamic energy benchmarking framework for detecting irregular energy consumption and improving energy efficiency. To simulate ANN-based MPC controllers for HVAC systems to evaluate potential cost savings compared with static control systems. To test and compare different ML classifiers—such as Random Forest, Logistic Regression, and Support Vector Machines (SVM)—for anomaly detection and fault diagnosis. The study achieved high predictive accuracy, reporting 94.5% accuracy for Random Forest, outperforming SVM and basic Decision Tree models.

The research of Singh et al. (2023) provides an advanced, integrative framework for predictive maintenance that combines the diagnostic capabilities of machine learning with the adaptability of MPC and the analytical depth of dynamic energy benchmarking. By achieving over 94% fault classification accuracy and demonstrating up to 83% cost savings, the study highlights the strong potential of data-driven maintenance strategies in transforming HVAC operations from reactive to intelligent and autonomous systems.

While the paper offers a robust foundation, further studies should focus on expanding datasets, improving interpretability, and implementing real-time embedded predictive maintenance systems suitable for large commercial or industrial environments.

2.4 Fault detection for air conditioning system using machine learning [4]

Sulaiman, N., Abdullah, M., Abdullah H., Zainudin, M., Yusop, A., 2020, IAES, Vol. 9 Issue 1

This review focuses on the critical domain of Fault Detection and Diagnosis (FDD) for Heating, Ventilation, and Air Conditioning (HVAC) systems, with a specific emphasis on data-driven machine learning (ML) approaches. The primary output of a successful FDD system is the accurate classification of the system's state (normal or faulty) and the identification of the specific fault type, enabling proactive maintenance and preventing energy waste. Research in HVAC FDD can be categorized into three main methodological strands, highlighting the field's evolution and key tensions: Data-Driven Techniques: This is the most prominent contemporary approach. It uses historical operational data to train models without requiring deep physical knowledge, thus reducing modeling complexity. This category includes methods like: Machine Learning (ML) Algorithms: This includes classifiers like Support Vector Machines (SVM) (Li et al., 2019), Artificial Neural Networks (ANN) (Najafi et al., 2012), and Deep Learning (Heo & Lee, 2019). The research by Sulaiman et al. had several key objectives: Synthesize the progression of FDD techniques from model-based to data-driven ML methods. Compare the performance of different ML classifiers as reported in key studies. Identify the research gap concerning system-wide FDD, as addressed by Sulaiman et al. (2020). Highlight the comparative effectiveness of different algorithms in a specific application context.

The core theories revolve around the machine learning algorithms themselves and the performance metrics used for evaluation. Support Vector Machine (SVM): A classifier that finds the optimal hyperplane to separate different classes of data in a high-dimensional space. Li et al. (2019) combined it with PCA for efficient HVAC FDD. Deep Learning: A subset of ML using neural networks with many layers (hence "deep") to progressively extract higher-level features from raw input. Accuracy: The proportion of total correct predictions (both true positives and true negatives) among the total number of cases examined. Precision: The proportion of correctly identified positive predictions among all instances predicted as positive. It measures the model's reliability.

This review establishes that data-driven ML methods are the state-of-the-art for HVAC FDD. The work of Sulaiman et al. (2020) is significant for its system-wide approach and direct comparison of classifiers, providing a valuable benchmark and highlighting MLP's potential for accurate, system-level fault detection. Future research should focus on validating these findings in real-world, large-scale installations and exploring hybrid models for improved robustness.

2.5 Predictive Maintenance in Building Facilities: A Machine Learning Approach. **Sensors** [5]

Bouabdallaoui, Y., Lafhaj, Z., Pascal, Y., Ducoulombier, L., Bennadji, B., 2021, MDPI, Vol. 21 Issue 4

This review synthesizes the concept of predictive maintenance (PdM) as a transformative approach for building facility management. The literature establishes a critical evolution in maintenance paradigms, distinguishing between reactive corrective maintenance, scheduled preventive maintenance, and the data-driven approach of predictive maintenance. PdM is specifically defined as a strategy that uses condition monitoring data to forecast future machine health, aiming to predict the timing, location, and nature of potential failures (Bouabdallaoui et al., 2021). The feasibility of PdM is fundamentally anchored in the diverse data sources available in modern buildings. Key sources include Building Automation Systems (BAS) for real-time operational data, Internet of Things (IoT) sensors for parameters like vibration and temperature, and systems like Computerized Maintenance Management Systems (CMMS) and Building Information Modeling (BIM) which provide historical records and rich semantic data, respectively. A significant and useful output from recent research is the application of unsupervised deep learning models, specifically autoencoders, for fault prediction. These models learn the "normal" operating patterns of a system from unlabeled data and flag deviations as anomalies, which is crucial given the general scarcity of labeled fault data in facility management. This study aims to outline the transition from traditional maintenance strategies to data-driven PdM in facility management. It seeks to describe the core components and data requirements

of a modern PdM framework and to identify the advantages of using unsupervised deep learning models for anomaly detection. Finally, it highlights the practical challenges and research gaps in implementing PdM, as revealed through empirical case studies. The key formula for this approach is the calculation of the Anomaly Score using Root Mean Square Error (RMSE). The RMSE between the input vector X and the reconstructed output vector $\hat{X}$, is calculated as follows:

$$RMSE(X, \hat{X}) = \sqrt{\frac{\sum_{i=1}^{n}(x_i - \hat{x_i})^2}{n}}$$

A threshold is set based on validation, and if the RMSE exceeds this threshold, an anomaly alert is triggered, signaling a potential fault.

The study includes a proposed framework which represents a machine learning approach adapted to the building context. The framework is composed of five steps: data collection, data processing, model development, fault notification and model improvement.

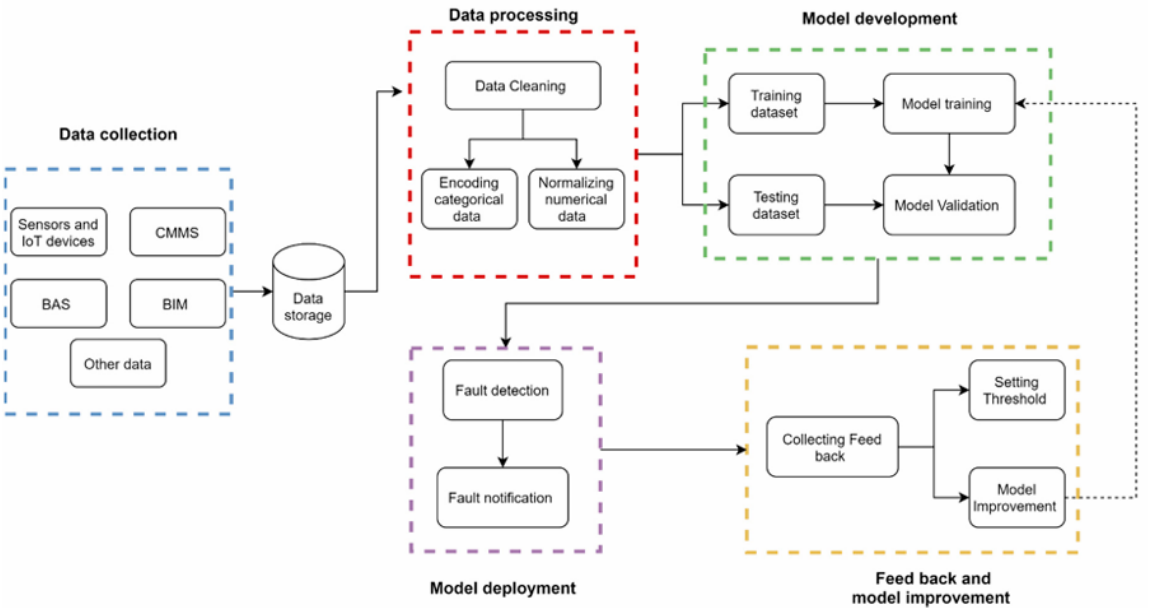


Figure 2. The proposed framework architecture.

2.6 Machine Learning Algorithms for Predictive Maintenance in HVAC Systems [6]

Sharma, V., Mistry, V., 2023, Journal of Scientific and Engineering Research, Vol. 10 Issue 11

The growing need for energy-efficient and reliable building systems has pushed Heating, Ventilation, and Air Conditioning (HVAC) maintenance away from reactive approaches and into data-driven, predictive strategies. The recent study of Sharma and Mistry (2023) provides a comprehensive overview of how machine learning algorithms can fundamentally change the way HVAC systems are monitored, maintained, and optimized. Their work emphasizes a shift from waiting for system failures to occur, to predicting potential issues in advance using data such as temperature readings, system load, airflow, vibration, and power consumption. This approach reflects a broader trend in current literature, where researchers explore intelligent maintenance systems that reduce downtime, prevent unexpected failures, and enhance energy efficiency in commercial buildings. Their discussion introduces key machine learning themes widely used in recent HVAC research:

1. Supervised Learning (Decision Trees, Random Forests, Support Vector Machines) These algorithms classify HVAC conditions as either normal or faulty based on labeled training data. Decision trees generate interpretable rule-based models, while random forests combine multiple trees to improve accuracy and reduce overfitting.
2. Deep Learning (ANN, CNN, RNN) These models capture complex nonlinear relationships in HVAC data. CNNs detect faults using image-based sensory data (e.g., thermal images), while RNNs process time-series readings to understand normal vs. degrading performance over time.

Sharma and Mistry organized their discussion around several clear research objectives that align with what most predictive maintenance studies aim to accomplish: Identify machine learning algorithms applicable to predictive HVAC maintenance.

Their work surveys the strengths and limitations of each major algorithm category. Outline a general workflow for implementing predictive maintenance using ML. The paper describes steps such as data collection, preprocessing, feature selection, model training, model evaluation, and real-world system integration. Show the practical impact of ML-based predictive maintenance. They summarize real case studies demonstrating energy savings and reduced equipment runtime for large-scale facilities (e.g., a shopping center in Canada with 509,612 ft² of space). Discuss emerging industry trends that support ML-based HVAC systems. These include cloud platforms, IoT sensors, edge devices, AWS IoT rules engines, and real-time anomaly detection services. Sharma and Mistry outline several diagrams and workflows that have become standard in predictive HVAC maintenance literature.

Figure 2: A standard workflow of applying anomaly detection approach on HVAC data

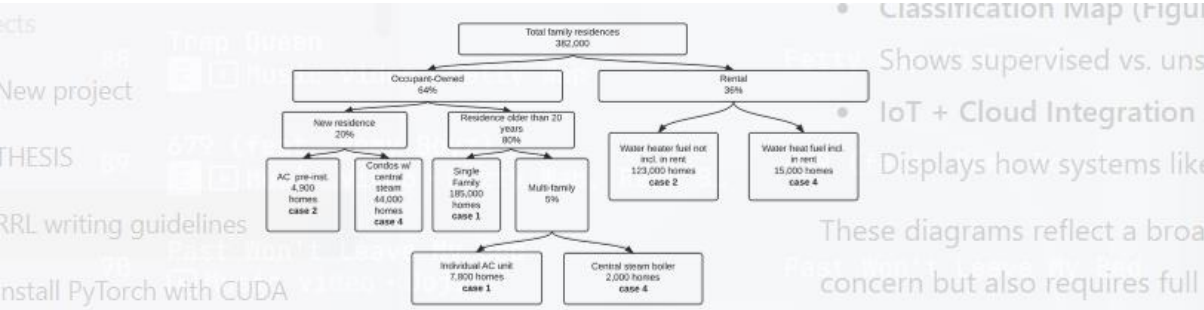


Figure 3: Decision tree for estimating amount of HVAC units affected by each principal-agent problem

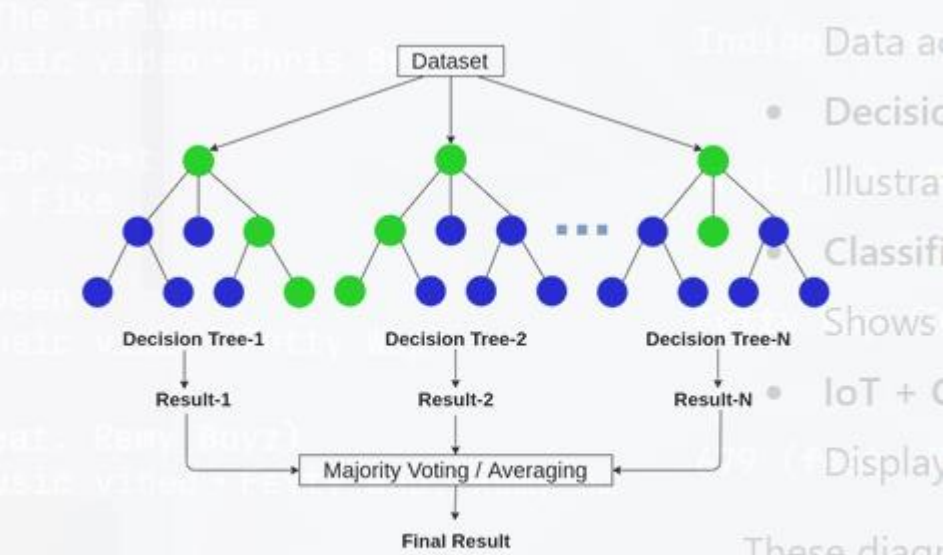


Figure 4: Classification of machine learning algorithms

The work of Sharma and Mistry (2023) synthesizes major trends in HVAC predictive maintenance by evaluating a wide range of machine learning techniques and their potential to improve reliability, energy efficiency, and operational cost. Their paper supports the broader shift toward intelligent building systems where ML acts as the backbone for early fault detection, system optimization, and automated decision-making. While challenges remain—especially in dataset quality, real-time integration, and scalable deployment—the current literature clearly demonstrates that predictive maintenance powered by machine learning is emerging as a core component of sustainable and smart building management.

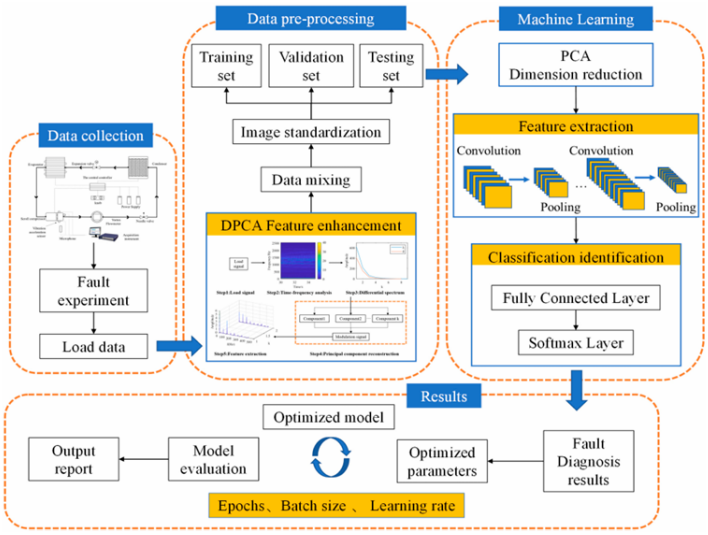
2.7 Research on Fault Diagnosis Strategy of Air-Conditioning Systems Based on DPCA and Machine Learning [7]

Song, Y., Ma, Q., Zhang, T., Li, F., Yu, Y., 2023, MDPI, Vol. 11 Issue 4

This review examines a novel hybrid approach for fault diagnosis in air-conditioning systems, which is critical for energy conservation and operational reliability. This RRL aims to synthesize the core contributions of the study by Song et al. (2023). It will delineate the operational principles of the DPCA

feature enhancement method and the architecture of the VGG-PCA model. Furthermore, it will analyze the comparative performance of this hybrid strategy against other machine learning models, using key metrics such as correct rate and running time. Finally, it will consolidate the practical guidelines provided for optimizing model parameters, which serves as a valuable reference for future implementations and research.

The study also includes a fault diagnosis strategy structure of air conditioning systems. A fault test was carried out to collect operation data. Then, the data are preprocessed and feature strengthened. To reduce the impact of random error on model training and testing, the loaded data are mixed and standardized. The data are randomly divided into training sets, verification sets and test sets.



In conclusion, the research by Song et al. (2023) presents a significant advancement in the field of data-driven fault diagnosis for air-conditioning systems. The study convincingly demonstrates that a strategy combining advanced signal processing (DPCA) with a sophisticated deep learning model (VGG-PCA) can yield superior performance. The DPCA method proves highly effective in enhancing fault features, leading to a substantial 16.38% increase in correct rate compared to using raw time-domain data. The VGG-PCA model itself showcases exceptional diagnostic capability, outperforming other CNN models by over 17% in accuracy while simultaneously reducing run-time by nearly 70%. The provision of a detailed parameter optimization strategy further adds to the practical value of this work, offering a clear roadmap for implementation. Future research directions, as suggested by the authors, include applying this hybrid approach to a broader set of faults and integrating the PCA

method with other advanced machine learning models to push the boundaries of diagnostic efficiency and accuracy further.

2.8 Predictive maintenance magnetic sensor using random forest method [8]

Aji, A., Sashiomarda, J., Handoko, D., 2020, Journal of Physics: Conference

Aji, Sashiomarda, & Handoko, (2020). The literature on predictive maintenance using magnetic sensors highlights the use of earth magnetic field data for real-time equipment condition monitoring, particularly in geophysical sensor networks like those managed by BMKG in Indonesia. Predictive maintenance (PdM) techniques leverage sensor data to forecast equipment failures before they occur, thus preventing unexpected downtime and reducing maintenance costs. Among machine learning methods, the random forest algorithm stands out for its effectiveness in PdM, as it aggregates multiple decision trees to improve prediction accuracy. Prior studies demonstrate the application of random forests in various industrial contexts, including semiconductor manufacturing and wind turbine maintenance, where historical sensor data are used to build predictive models. Specifically, the total component of the geomagnetic field (F) from magnetic sensors can be processed and analyzed to detect deviations indicating sensor degradation or imminent failure. Recent research shows that predictive models using random forest can achieve high accuracy (RF score up to 0.98) and low errors (MAE around 0.83) in forecasting maintenance needs for magnetic sensor equipment. These models enable maintenance scheduling based on data-driven thresholds, enhancing sensor reliability and data quality for continuous observation systems.

2.9 Predictive maintenance of electromechanical systems based on enhanced generative adversarial neural network with convolutional neural network. [9]

Abood, A., Nasser, A., Al-Khazraji, H., 2022, IIETA, Vol. 27, Issue 6

Abood, A. M. et al. (2023). Predictive maintenance (PdM) has gained significant attention as a cost-effective strategy to prevent equipment breakdowns and minimize production losses in industrial systems. It leverages deep learning (DL) techniques, particularly convolutional neural networks (CNN) and generative adversarial networks (GAN), to analyze vast amounts of data generated by sensor technologies for early fault detection in electromechanical systems (Abood, Nasser, & Al-Khazraji, 2023). The hybrid CNN-conditional GAN (CGAN) model proposed integrates CNN's feature extraction capability with CGAN's data generation and classification strength, achieving improved accuracy and reduced complexity in multiclass fault diagnosis, demonstrated on an asynchronous motor fault dataset with superior performance metrics (F-score of 100) compared to standalone CGAN and other DL models. This integration utilizes a deep learning framework that enhances fault prediction robustness by effectively classifying fault states and predicting machine health, offering promising advancements in PdM applications for industrial motors

2.10 Machine Learning: Algorithms, Real-World Applications and Research Directions [10]

Sarker, I., 2021, SN Computer Science, Vol. 2 Issue 160

Supervised learning has emerged as one of the most established branches of machine learning because of its ability to learn from labeled datasets and generate reliable predictions for classification and regression tasks. In the study by Sarker (2021), supervised learning is presented as a central technique where models learn a mapping between input variables and known target outputs, enabling them to generalize and perform accurate predictions in real-world applications. This approach has been widely applied across domains such as medical diagnosis, financial forecasting, industrial equipment monitoring, and intelligent decision-making systems. The article provides a thorough discussion of various supervised learning algorithms, their mechanisms, advantages, and practical challenges—offering an important foundation for understanding how predictive models are developed and deployed.

The study discusses how the machine learning algorithms is mainly divided into four categories. The only type that is useful for this following research is the following:

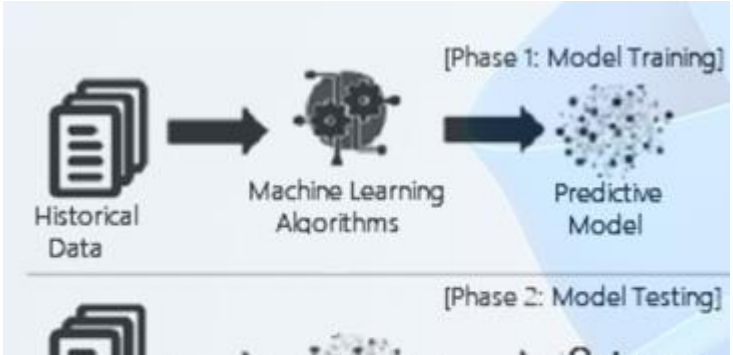
3. *Supervised*: Supervised learning is typically the task of machine learning to learn a function that maps an input to an output based on sample input-output pairs. It uses labeled training data and a collection of training examples to infer a function. Supervised learning is carried out when certain goals are identified to be accomplished from a certain set of inputs, i.e., a task-driven approach. The most common supervised tasks are “classification” that separates the data, and “regression” that fits the data. For instance, predicting the class label or sentiment of a piece of text, like a tweet or a product review, i.e., text classification, is an example of supervised learning.

The study suggests that to build effective models in various application areas different types of machine learning techniques can play a significant role according to their learning capabilities, depending on the nature of the data discussed earlier, and the target outcome. These machine learning types are then summarized in the table that they provided.

Table 1 Various types of machine learning techniques with examples

Learning type	Model building	Examples
Supervised	Algorithms or models learn from labeled data (task-driven approach)	Classification, regression
Unsupervised	Algorithms or models learn from unlabeled data (Data-Driven Approach)	Clustering, associations, dimensionality reduction
Semi-supervised	Models are built using combined data (labeled + unlabeled)	Classification, clustering
Reinforcement	Models are based on reward or penalty (environment-driven approach)	Classification, control

This study includes a detailed discussion about one of the tasks of supervising learning namely classification analysis which this research is based upon.



Classification Analysis

Classification is regarded as a supervised learning method in machine learning, referring to a problem of predictive modeling as well, where a class label is predicted for a given example. Mathematically, it maps a function (f) from input variables (X) to output variables (Y) as target, label or categories. For example, spam detection such as “spam” and “not spam” in email service providers can be a classification problem. In the following, the researchers will summarize the common classification problems that will also useful for this research:

K-nearest neighbors (KNN): K-Nearest Neighbors (KNN) [9] is an “instance-based learning” or non-generalizing learning, also known as a “lazy learning” algorithm. It does not focus on constructing a general internal model; instead, it stores all instances corresponding to training data in n-dimensional space. KNN uses data and classifies new data points based on similarity measures (e.g., Euclidean distance function)

$$d = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

Classification is computed from a simple majority vote of the k nearest neighbors of each point. It is quite robust to noisy training data, and accuracy depends on the data quality. The biggest issue with

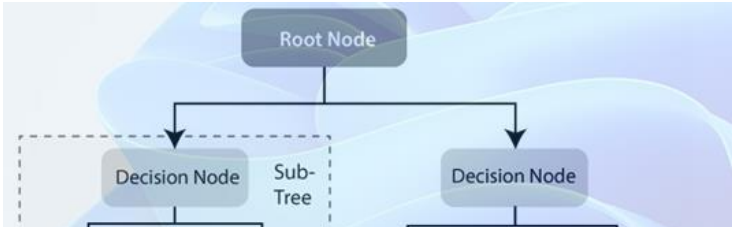
KNN is to choose the optimal number of neighbors to be considered. KNN can be used both for classification as well as regression.

Support Vector Machine (SVM): In machine learning, another common technique that can be used for classification, regression, or other tasks is a support vector machine (SVM). In high- or infinite-dimensional space, a support vector machine constructs a hyper-plane or set of hyper-planes. Intuitively, the hyper-plane, which has the greatest distance from the nearest training data points in any class, achieves a strong separation since, in general, the greater the margin, the lower the classifier's generalization error. It is effective in high-dimensional spaces and can behave differently based on different mathematical functions known as the kernel. Linear, polynomial, radial basis function (RBF), sigmoid, etc., are the popular kernel functions used in SVM classifier [82]. However, when the data set contains more noise, such as overlapping target classes, SVM does not perform well.

Decision Tree (DT): Decision tree (DT) is a well-known non-parametric supervised learning method. DT learning methods are used for both the classification and regression tasks [82]. By sorting down the tree from the root to some leaf nodes, as shown in Fig.4, DT classifies the instances. Instances are classified by checking the attribute defined by that node, starting at the root node of the tree, and then moving down the tree branch corresponding to the attribute value. For splitting, the most popular criteria are "gini" for the Gini impurity and "entropy" for the information gain that can be expressed mathematically as:

$$\text{Entropy: } H(x) = -\sum_{i=1}^{n} p(x_i)\ log_2\ p(x_i)$$

$$\text{Gini: } E = 1 - \sum_{i=1}^{c} p_i^2$$



Random Forest (RF): A random forest classifier is well known as an ensemble classification technique that is used in the field of machine learning and data science in various application areas. This method uses “parallel ensembling” which fits several decision tree classifiers in parallel, as shown in Fig.5, on different data set sub-samples and uses majority voting or aver ages for the outcome or final result. It thus minimizes the over-fitting problem and increases the prediction accuracy and control. Therefore, the RF learning model with multiple decision trees is typically more accurate than a single decision tree-based model. To build a series of decision trees with controlled variation, it combines bootstrap aggregation (bagging) and random feature selection. It is adaptable to both classification and regression problems and fits well for both categorical and continuous values.

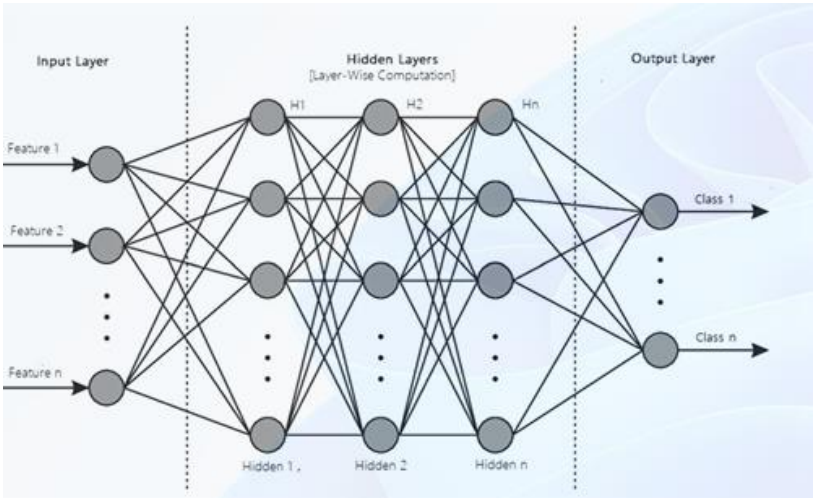
Extreme gradient boosting (XGBoost): Gradient Boosting, like Random Forests above, is an ensemble learning algorithm that generates a final model based on a series of individual models, typically decision trees. The gradient is used to minimize the loss function, similar to how neural networks use gradient descent to optimize weights. Extreme Gradient Boosting (XGBoost) is a form of gradient boosting that takes more detailed approximations into account when determining the best model. It computes second-order gradients of the loss function to minimize loss and advanced regularization (L1 and L2) [82], which reduces over-fitting, and improves model generalization and performance. XGBoost is fast to interpret and can handle large-sized datasets well.

A special type of supervised learning is also presented by the study which is called Deep Learning. *Deep Learning* is part of a wider family of artificial neural networks (ANN)-based machine learning

approaches with representation learning. Deep learning provides a computa tional architecture by combining several processing layers, such as input, hidden, and output layers, to learn from data. The main advantage of deep learning over traditional machine learning methods is its better performance in sev eral cases, particularly learning from large datasets.

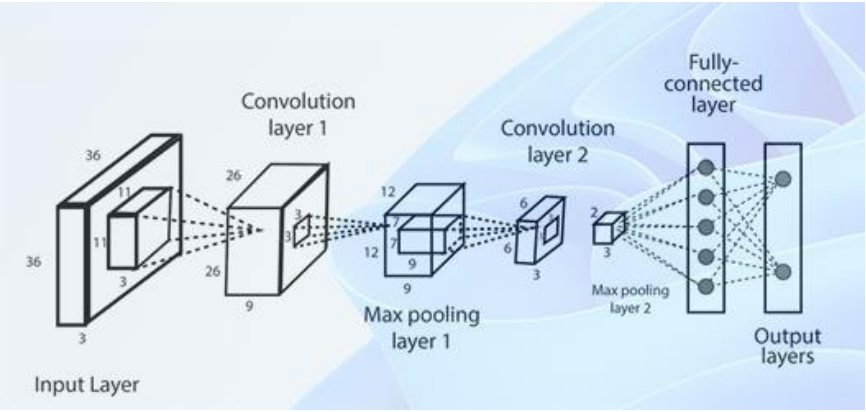
The most common deep learning algorithms are: Multi-layer Perceptron (MLP), Convolutional Neural Network (CNN) which will be used for this research. In the following, the study will discuss various types of deep learning methods that ca be used to build effective data-driven models for various purposes.

MLP: The base architecture of deep learning, which is also known as the feed-forward artificial neural network, is called a multilayer perceptron (MLP). A typical MLP is a fully connected network consisting of an input layer, one or more hidden layers, and an output layer. Each node in one layer connects to each node in the following layer at a certain weight. MLP utilizes the “Backpropagation” technique, the most “fundamental building block” in a neural network, to adjust the weight values internally while building the model. MLP is sensitive to scaling features and allows a variety of hyperparameters to be tuned, such as the number of hidden layers, neurons, and iterations, which can result in a computationally costly model.



CNN or ConvNet: The convolution neural network (CNN) enhances the design of the standard ANN, consisting of convolutional layers, pooling layers, as well as fully connected layers. As it takes the advantage of the two-dimensional (2D) structure of the input data, it is typically broadly used in several

areas such as image and video recognition, image processing and classification, medical image analysis, natural language processing, etc. While CNN has a greater computational burden, without any manual intervention, it has the advantage of automatically detecting the important features, and hence CNN is considered to be more powerful than conventional ANN. A number of advanced deep learning models based on CNN can be used in the field, such as AlexNet, Xception, Inception, Visual Geometry Group (VGG), ResNet, etc.



The study by **Sarker (2021)** provides a comprehensive and accessible overview of supervised learning algorithms, describing how they function, their theoretical bases, and their practical roles in modern machine learning applications. By outlining both the advantages and challenges of each algorithm, the article serves as a valuable foundation for researchers exploring classification and regression tasks. Despite significant progress, the field continues to face challenges related to noisy data, high-dimensional modeling, and large-scale deployment—making supervised learning an active and evolving area of research.

CHAPTER III: METHODOLOGY

3.1 DATA COLLECTION

3.1.1 RESEARCH DESIGN OVERVIEW

This study employs an Experimental Research Design to develop a predictive maintenance (PdM) model for evaporator coils in split-type air conditioning systems installed in the College of Engineering Building, Faber Hall, and the SBM-AVR. The design is experimental because the researchers implement a controlled, systematic, and instrumented data collection procedure using calibrated sensors, synchronized microcontroller nodes, and repeated measurement trials under uniform conditions.a standardized hardware setup, conducts sensor calibration experiments, and uses strict measurement protocols to ensure consistent and reproducible data. Each AC unit undergoes controlled trials per scheduled visit, and all sensors BME280, DS18B20, ACS712, ZMPT101B, and ESP32-CAM collect data simultaneously using an ESP-NOW synchronized timestamp. This controlled setup ensures that the system parameters are measured scientifically and uniformly.During the 6-month monitoring period, the researchers observe naturally occurring behaviors of evaporator coils temperature variations, ice formation,noise and Refrigerant leak conditions but measure these behaviors using an experimentally installed multi-sensor system. The systematic and repeated measurement process enables the identification of correlations between operating conditions and early signs of evaporator coil abnormalities.The study aims to evaluate and compare machine learning models (CNN, Random Forest, XGBoost, k-NN, RBK SVM and DNN) using data produced from controlled trials. The reliability of this comparison depends on the consistent, experiment-based collection of data rather than purely passive observation. Using experimental procedures ensures that the predictive maintenance model is trained on accurate, validated, and standardized sensor data, allowing fair and scientific evaluation of model performance through training, validation, and testing splits.

3.1.2 SOURCES OF DATA

Features	Data Type	Source (Sensors)	Purpose
Ambient Temperature	Continuous	BME280	Measures environmental temperature that influences AC load and system behavior
Humidity	Continuous	BME280	Monitors moisture levels that contribute to coil icing and airflow restriction
Pressure	Continuous	BME280	Tracks atmospheric pressure to support the overall thermal profiling of the environment
Inlet Air Temp	Continuous	DS18B20	Records temperature entering the indoor
Outlet Air Temp	Continuous	DS18B20	Records temperature leaving the indoor unit, enabling evaluation of cooling performance
Coil Surface Temp	Continuous	DS18B20	Detects abnormal coil temperature that indicate frost, fouling, or refrigerant issues
Compressor Current Draw	Continuous	ACS712	Monitors compressor electrical load to detect inefficiencies, strain, and early faults
Fan Motor Current	Continuous	ACS712	Measures fan electrical behavior to identify airflow anomalies or motor degradation

Supply Voltage	Continuous	ZMPT101B	Captures voltage fluctuations affecting compressor and fan performance
Frost Build Up	Nominal	ESP32-CAM and CNN	Identifies presence or absence of frost via image-based classification
Coil / Fin Condition	Nominal	ESP32-CAM and CNN	Detects coil cleanliness, fin blockage, and fouling through image analysis

3.1.3 SAMPLING

This study employs purposive sampling, selecting air conditioning units that best represent real-world usage conditions in academic environments. A maximum of 50 split-type air conditioning units will be monitored across three primary locations: the SBM-AVR, the College of Engineering, and Faber Hall.

1.SBM-AVR Facilities Units in the AVR are selected due to their operational hours, and tendency to accumulate dirt and humidity.

2.College of Engineering Building faculty rooms, and laboratories are included because they are easy to access and allow uniform weekly measurement. 3.Faber Hall Additional units are selected 1st floor only to complete the needed sample size while still maintaining consistent monitoring procedures.

3.1.4 HARDWARE USED

Different sensors alongside microcontrollers are used to collect different types of data. There will be 3 groups of sensors each having one microcontroller to power these sensors.

Ambient and Temperature Node

- Microcontroller - ESP32C3
- Ambient Temperature, Humidity, and Pressure - BME280
- Inlet Air Temp - DS18B20
- Outlet Air Temp - DS18B20
- Coil Surface Temp - DS18B20

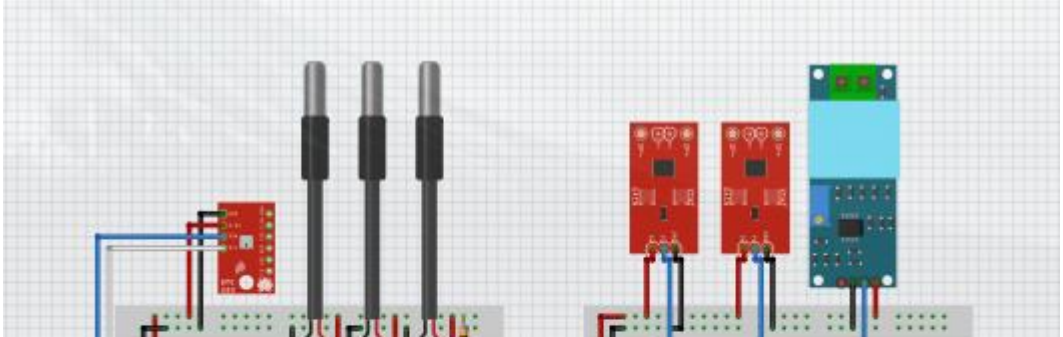
Electrical Node

- Microcontroller - ESP32C3
- Compressor Current Draw - ACS712
- Fan Motor Current - ACS712
- Supply Voltage - ZMPT101B

Frost Node

- Microcontroller - ESP32CAM
- Coil / Fin Condition - ESP32-CAM
- Frost / Build Up - ESP32-CAM

All ESP32 shares data through ESP Now Communication where one ESP32 serves as the Master and All ESPs take measurements at the same timestamp. The Master collects all data + timestamps, then stores or sends it to a PC/server.



3.1.5 DATA GATHERING PROCEDURE

The data collection procedure begins with preparing and scheduling the units, where about 50 units are planned for testing each week over a period that can extend up to six months. Once the schedule is set, each unit proceeds to the data gathering stage, where it undergoes 10 measurement trials and is categorized as either Normal or Abnormal based on criteria validated by experts. After this, the data labelling phase is carried out in two steps: first, the images collected from each unit are labelled according to the established criteria, and second, the measurement data is assigned its final label by matching it with the image classification, determining whether it is Normal or Abnormal.

3.1.6 VALIDITY AND RELIABILITY MEASURES

Validity Experiment

This section details the rigorous, pre-deployment validation and calibration protocols established to ensure the instrument validity of the multi-sensor array. The reliability of any

predictive model is fundamentally dependent on the quality of its training data; therefore, a robust validation phase is critical to the experimental design outlined in this study.

Sensor Validation Experiment Design

Before commencing the 6-month data gathering procedure 2, a separate set of validation experiments will be conducted. The objective is to quantify and, where possible, correct the inherent manufacturing tolerances and measurement errors of each sensor. Commercial-off-the-shelf (COTS) sensors, while cost-effective, possess stated accuracy bounds (e.g., +-0.5°C for the DS18B20) that may be insufficient for developing a high-fidelity predictive model.

This protocol will establish a traceable, sensor-specific Calibration Correction Function (CCF) for every sensor deployed in the study.

The research design specifies monitoring a maximum of 50 air conditioning units. The hardware schematic details one BME280 (Ambient Temperature, Humidity, Pressure), three DS18B20s (Inlet Air Temp, Outlet Air Temp, Coil Surface Temp), two ACS712s (Compressor Current Draw, Fan Motor Current), and one ZMPT101B (Supply Voltage) for each unit. This results in a total sensor pool of 350 individual COTS sensors that require validation (50 BME280, 150 DS18B20, 100 ACS712, 50 ZMPT101B). Calibrating 350 sensors individually is logistically prohibitive.

Therefore, the validation procedures described in the following sub-sections are explicitly designed for batch calibration, where multiple sensors of the same type are tested simultaneously against a single, superior-grade reference instrument.

The general validation procedure is as follows:

1. **Unique Identification**: Every sensor module will be assigned a unique identifier (UID). For DS18B20 sensors, this will be their native 64-bit 1-Wire serial code. For the BME280, ACS712, and ZMPT101B modules, a human-readable UID (e.g., BME-01 to BME-50) will be physically labeled on the module and digitally associated with the master ESP32 node it is connected to.

2. **Batch Testing**: Sensors will be tested in batches within a controlled environment (e.g., thermal chamber, stabilized liquid bath, or electrical test jig).

3. **Reference Comparison**: Measurements from all sensors in the batch will be recorded and correlated against a traceable, high-precision reference instrument (e.g., a Fluke reference thermometer or a high-precision power analyzer).

4. **CCF Derivation**: For each UID, a CCF will be derived. This may range from a simple numerical offset to a non-linear polynomial equation.

5. **Database Storage**: All UIDs and their corresponding CCF coefficients (e.g., slope, intercept, polynomial terms) will be stored in a central calibration database (e.g., a JSON or CSV file). This allows data preprocessing scripts to automatically apply the correct CCF to the raw data from any given sensor UID, programmatically converting raw, error-prone sensor readings into corrected, validated physical units.

Table 3.1: Sensor Validation Summary and Reference Standards

Sensor	Parameter(s) Measured	Traceable Reference Instrument	Validation Environment	Calibration Method
BME280	Ambient Temp, Humidity	Calibrated Thermo-Hygrometer (e.g., Vaisala HMP 35A)	Sealed Environmental Chamber	2-Point Linear Regression
BME280	Atmospheric Pressure	Certified METAR Station Data	Open-Air, Stable Lab Env.	Single-Point Altitude Correction & Offset
DS18B20	Air/Coil Temperature	Fluke Hart Scientific Reference Thermometer	Circulating, Stabilized Liquid Bath	3-Point Linear Regression
ACS712	Compressor/Fan Current	Fluke 115 True-RMS Multimeter	AC Test Jig with Known Resistive Loads	2-Step (Offset + Linear Regression)
ZMPT101B	Supply Voltage	Fluke 115 True-RMS Multimeter	Variable AC Transformer (Variac)	Multi-Point 3rd-Order Polynomial Regression

BME280 (Temperature, Humidity, Pressure) Validation

- **Temperature & Humidity**: A calibrated, traceable thermo-hygrometer (e.g., Vaisala HMP 35A) will serve as the reference standard for temperature and relative humidity.
- **Pressure**: A professional-grade barometer or, more practicably, the corrected data from a local, certified METAR (Meteorological Aerodrome Report) station will be used. This data is professionally maintained and provides a reliable baseline for barometric pressure.
- **Environment**: A small, sealed environmental chamber will be used. Known humidity levels will be generated using saturated salt solutions (e.g., Sodium Chloride for ~75% RH, Magnesium Chloride for ~33% RH), which is a standard technique for hygrometer calibration. Temperature will be controlled using a thermoelectric (Peltier) element.

A critical factor for BME280 temperature validation is compensating for systemic heat. The BME280 datasheet warns that the internal temperature sensor reading is "typically above ambient temperature" due to "sensor element self-heating" and, more significantly, heat from the PCB. In this study's "Thermal Node" design, the BME280 sensor is co-located on the same PCB as an ESP32C3 microcontroller, which is a significant heat source. Therefore, calibrating the BME280 sensor in isolation is insufficient. The validation must be performed on the fully assembled "Thermal Node" to characterize the systemic thermal offset induced by the ESP32 and other components. The resulting CCF will correct for both the sensor's inherent error and this localized heat contamination.

Temperature and Humidity Validation Procedure

1. **Assembly**: The Thermal Node boards are fully assembled and placed inside the environmental chamber. The reference thermo-hygrometer probe is placed in the center of the sensor batch.

2. **Temperature Calibration (Multi-Point)**:

- The chamber temperature is stabilized at three setpoints (e.g., 15°C, 25°C, and 35°C) to cover the expected operating range.
- At each setpoint, the system is left for 30 minutes to achieve thermal equilibrium, accounting for sensor stabilization time.
- Data is then logged for 10 minutes from all 50 BME280s and the reference instrument.
- A linear regression is performed for each sensor's UID by comparing its raw reading (Traw) to the reference reading (Tref), generating a unique Temp_CCF in the form of Tcorrected = m x Traw + c

3. **Humidity Calibration (2-Point):**

- A saturated Sodium Chloride (NaCl) solution is placed in the chamber, creating a stable ~75% RH environment. The system is stabilized, and 10 minutes of data are logged.
- The process is repeated using a Magnesium Chloride ($MgCl_2$) solution, which creates a stable ~33% RH environment.
- A 2-point linear regression is performed for each sensor, comparing its raw RH reading (compensated using its newly derived Temp_CCF) against the reference RH. This generates a unique Humid_CCF.

Barometric Pressure Validation Procedure

1. **Data Collection**: "Thermal Node" boards are run for 1 hour in a stable indoor laboratory environment (e.g., on a workbench).

2. **Reference Acquisition**: During this 1-hour window, the official sea-level-corrected pressure (QNH) is acquired from the nearest certified METAR station.

3. **Altitude Correction**: The METAR pressure is corrected for the laboratory's specific altitude above sea level using the barometric formula. This calculation converts the sea-level pressure (QNH) to the true local atmospheric pressure (QFE).

DS18B20 (Temperature) Validation

- **Reference:** A traceable, high-precision digital reference thermometer (e.g., a "Fluke Hart Scientific" standard sensor or an ASTM 117C thermometer) with a resolution of at least 0.01°C
- **Environment:** A temperature-controlled liquid bath, such as an oil bath for thermal stability or a distilled water bath.
- **Critical Component:** A liquid circulation pump (e.g., an aquarium pump) is **mandatory**. This component ensures vigorous circulation, which eliminates thermal stratification within the bath and guarantees that all sensor probes and the reference probe are exposed to a homogenous thermal environment.

The DS18B20 sensor's primary feature, the 1-Wire bus protocol, is a significant advantage for this validation. The protocol allows "multiple DS18B20s to function on the same 1-Wire bus," and each sensor is already uniquely identifiable via its "unique 64-bit serial code". This avoids the need for manual UID labeling and enables a highly efficient mass calibration. A single "calibration rig" can be constructed by connecting all 150 sensors in parallel to a single master microcontroller. A script can then iterate through all 64-bit IDs on the bus, request a temperature conversion from each, and log the ID and its corresponding raw reading, fully automating the mass calibration process.

Multi-Point Calibration Procedure

1. **Assembly:** All 150 waterproof DS18B20 sensor probes are bundled tightly with the high-precision reference probe. The bundle is submerged in the center of the liquid bath, and the circulation pump is activated.
2. **Point 1 (Ice Point ≈ 0°C):** The bath is filled with a properly prepared mixture of crushed ice and distilled water. The system is allowed to stabilize for 15 minutes.

3. Data Logging (Point 1): The master script polls all 150 sensors and the reference thermometer every 10 seconds for 10 minutes. The results (64-bit ID, Raw_Temp, Ref_Temp) are logged to a file.

ACS712 (Current) Validation

- Reference: A high-precision, True-RMS digital multimeter (DMM), such as a "Chauvin Arnoux 8335” or "Fluke 115". The True-RMS capability is non-negotiable, as compressor and fan motors are inductive loads and do not produce perfect sinusoidal AC waveforms.

- Loads: A set of high-power, purely resistive loads (e.g., 50 W, 200 W, 250 W incandescent bulbs or high-wattage power resistors).

- Source: A stable AC mains power source.

The ACS712 is an analog Hall-effect sensor, and its output is an analog voltage (typically centered at V cc/2), which is then read by the ESP32's Analog-to-Digital Converter (ADC). This sensor is notoriously noisy and prone to error, especially at low currents (0 A to 1 A). This low-current range is precisely where the "Fan Motor Current” is expected to operate, making calibration critical.

The validation must address two separate error sources:

1. **Zero-Current Offset:** The quiescent voltage output by the sensor when *no current* is flowing. This value "fluctuates A LOT" [27] and must be precisely calibrated for each sensor.

2. **Sensitivity (Gain):** The actual millivolt-per-Ampere (mV / A) caling factor, which also has a manufacturing tolerance.

Therefore, a two-step calibration procedure is required: first, determine the true zero point, and second, determine the gain (slope).

Zero-Current Offset Calibration

1. **Assembly**: All 50 "Electrical Node" boards are fully assembled.

2. **Procedure**: The ESP32 nodes are powered on (from their 3.3V/5V supply), but no AC load is connected to the high-voltage terminals of the ACS712 sensors.

3. **Data Logging**: A script runs on each ESP32 that reads the raw ADC value from its two ACS712 sensors (Compressor and Fan) 1000 times over 60 seconds. This averaging is necessary to filter out high-frequency noise.

4. **Offset Calculation**: The average of these 1000 readings is calculated. This value (e.g., 511.5) is the precise ADC_Zero_Offset for that specific sensor. This value is stored in the calibration database, linked to the sensor's UID. All future current

calculations will first subtract this offset from the raw ADC reading before any other processing. 1

ZMPT101B (Voltage) Validation

- **Reference**: A high-precision, True-RMS DMM (e.g., "FLUKE 115").
- **Source**: A Variable AC Transformer (Variac). This device is essential as it can output a stable, adjustable AC voltage across the full operational range (e.g., 50 V to 250 V)

The ZMPT101B module is often mistakenly assumed to be linear. However, analysis shows that the output waveform is "not really a replica" of the input and that the module introduces a phase shift. More importantly, rigorous calibration studies explicitly state that when correlating the input voltage to the ADC output, a simple linear fit is inaccurate. The "analysis of the polynomials shows that the **third-order polynomial gives the best relationship**".

To accurately "capture voltage fluctuations" as required by the research goals, a simple linear calibration is scientifically insufficient. The *best solution* is to adopt this peer-reviewed **3rd-Order Polynomial Regression** method for calibration.

A secondary but critical step is standardization. The module includes an onboard potentiometer (trimpot) for adjusting gain. If this trimpot is not standardized, the calibration is useless. The procedure from will be adopted: adjust the trimpot so the *maximum* expected voltage (250 V) maps to an ADC value *below* saturation (e.g., 640), providing necessary headroom.

3.1.7 DATA MANAGEMENT AND STORAGE

This section describes the comprehensive data management plan (DMP) for the 6-month experimental monitoring period. The plan is designed to ensure all data is securely stored, organized, and protected, adhering to the **FAIR** principles (Findable, Accessible, Interoperable, and Reusable) for academic research and data preservation.

Data Storage Architecture and Organization

The data flow and storage architecture is designed for robustness, scalability, and automated backup.

On-Site Ingest: The "Master" ESP32 at each AC unit will transmit its synchronized data payload and timestamp to a central "Ingest Server" (a dedicated PC or server) located in the laboratory via a private Wi-Fi network.

Primary On-Site Storage (NAS): The Ingest Server will write all incoming data directly into the logical directory structure (detailed in 7.1.1) hosted on a central **Network Attached Storage (NAS)** device. This NAS will serve as the primary, high-availability repository for all active research data.

Off-Site Cloud Storage: The NAS will be configured to perform automated, nightly synchronization with a secure, academic-grade cloud storage provider (e.g., AWS S3, Google Cloud, or a university-provided repository).

Logical Directory Structure

The methodology for this study explicitly defines a Data Preprocessing stage (Data Cleaning, Transformation, Reduction) that is separate from Data Gathering. This workflow is central to machine learning. Therefore, an "Advanced Project" folder structure will be adopted to logically separate data based on its processing stage.

This structure is critical. The raw folder will hold the original, immutable data. The interim folder will hold the data *after* it has been cleaned and calibrated using the CCFs from Section 6. The processed folder will hold the final, feature-engineered datasets used to train the ML models. This separation is the foundation of the "Keep Raw Data Raw" principle (see 7.3.1).

The proposed root directory structure on the NAS is as follows:

```
/AC_PdM_Research_Project/
│
├── 00_Documentation/
│   ├── README.md                (Project overview, contact info)
│   └── Data_Management_Plan.pdf  (This document)
│
├── 01_Sensor_Calibration/
│   ├── bme280_ccf.json          (Stores all 50 BME280 CCFs by UID)
│   ├── ds18b20_ccf.json         (Stores all 150 DS18B20 CCFs by 64-bit ID)
│   ├── acs712_ccf.json          (Stores all 100 ACS712 CCFs by UID)
│   └── zmpt101b_ccf.json        (Stores all 50 ZMPT101B poly-equations)
│
├── 02_Data/
│   ├── raw/                     (Immutable, original data from sensors)
│   │   ├── SBM-AVR_ACU-01/      (Folder per AC unit)
│   │   ├── COE_ACU-02/
│   │   └── ...
│   ├── interim/                 (Cleaned, calibrated, merged data)
│   └── processed/               (Feature-engineered, model-ready datasets)
│
├── 03_Code/
│   ├── 0_firmware/              (ESP32.ino sketches for nodes)
│   ├── 1_ingest/                (Python scripts on the Ingest Server)
│   ├── 2_preprocessing/         (Python scripts that convert raw -> interim)
│   └── 3_modeling/              (Jupyter/Python scripts for ML models)
│
└── 04_Outputs/
    ├── figures/                 (Plots, graphs for publication)
    ├── models/                  (Saved/pickled model files:.pth,.pkl)
    └── papers/                  (Thesis draft, conference papers)
```

Time-Series File Naming Convention

This project will generate thousands of data files from 50 locations over 6 months. A human-readable system like "Test data 2016.xlsx" is inadequate and leads to lost data. The file names *must* be consistent, descriptive, and machine parsable.

Breakdown:

- YYYYMMDDTHHmmSSZ: ISO 8601 timestamp indicating the *start* of the 10-trial measurement session. The Z denotes UTC (Universal Time Coordinated) to avoid all ambiguity related to local time or daylight saving.
- <Location>: Building code (e.g., SBM-AVR, COE, FABER).

- <UnitID>: Unique AC unit identifier (e.g., ACU-01, ACU-02).
- <TrialNum>: The trial number for that session (T01 through T10).
- <DataType>: RAW (for the CSV file containing all sensor data), IMG (for the corresponding ESP32-CAM image), or LOG (for system health/error logs).

Data Protection and Backup Strategy

Implementation of the "3-2-1" Redundancy Rule

The 6-month monitoring period represents a unique, non-repeatable data collection event. The loss of this data due to hardware failure, malware, or physical disaster (fire, flood, theft) would be catastrophic to the entire research project.

To mitigate this risk, the **"3-2-1 Backup Rule" will** be implemented as the minimum standard for data protection. This rule mandates:

- **3 copies** of the data.
- On **2 different media types**.
- With **1 copy off-site**.

Application of the 3-2-1 Rule:

1. **Copy 1 (Primary/Working):** The data residing on the primary **on-site NAS**. This is the "hot" data used for daily access by the Ingest Server and analysis scripts.
2. **Copy 2 (On-Site/Different Media):** A large-capacity **external USB hard drive** physically connected to the NAS. This represents the second media type. The NAS will be configured to perform a full, versioned backup to this external drive every week. This protects against NAS hardware failure or data corruption.
3. **Copy 3 (Off-Site/Cloud):** The NAS will perform an automated, encrypted, incremental **nightly backup** to a secure **cloud storage provider** (e.g., AWS S3, Google Cloud, or a

university-provided repository). This provides critical geographic redundancy and protects against a location-based disaster.

Backup Schedule and Recovery Protocol

- **Ingest Server -> NAS:** Data is written in near-real-time as it is collected.
- **NAS -> Cloud (Off-site):** A synchronization task will run nightly.
- **NAS -> External HDD (On-site):** A full backup task will run weekly.
- **Recovery Plan:** A formal recovery protocol will be documented. This plan will be tested *quarterly* by attempting to restore a random subset of data from the cloud backup to a new, isolated machine, verifying data integrity and backup functionality.
- **Security:** All cloud backups will be encrypted. Access to the cloud storage account and the local NAS will be protected by multi-factor authentication.

The "Keep Raw Data Raw" Immutable Data Policy

The methodology includes Data Cleaning, Handling Missing Data, and Data Transformation. These are, by definition, destructive operations. If these operations are performed on the original data files, the true raw data is permanently lost, and the research becomes irreproducible.

To prevent this, the **"Keep Raw Data Raw"** principle will be strictly enforced. This policy states that the original, raw data files are *sacrosanct* and must *never* be altered.

Policy Implementation:

1. **Read-Only:** All files and directories within the 02_Data/raw/ folder (see 7.1.1) will be set to **read-only** immediately after being successfully written by the Ingest Server.

2. **No Manual Edits: No changes or corrections** will ever be made directly to these files. Spreadsheet software (like Microsoft Excel) is explicitly forbidden for processing raw data, as it makes non-auditable changes and can corrupt data formats.
3. **Scripted Processing:** All preprocessing—including cleaning, applying the CCFs from the 01_Sensor_Calibration/ database, timestamping, and merging—will be performed **exclusively via documented scripts** (e.g., Python with Pandas, R).
4. **New File Generation:** These scripts will *read* from the .../raw/ directory, perform the transformations in memory, and **write new, clean files** to the .../interim/ directory.

Data Provenance and Reproducibility

This policy ensures a complete, auditable "chain of custody" for the data, known as *data provenance*. It provides a clear record tracking the data's origin (the sensor), its transformations (the scripts), and its final state (the processed dataset).

To complete this, all scripts used for ingestion (1_ingest/), preprocessing (2_preprocessing/), and modeling (3_modeling/) will be stored in the 03_Code/ directory and version-controlled using **Git**.

This combination of **immutable raw data** and **version-controlled processing scripts** guarantees **100% reproducibility**. At any point in the future, another researcher (or the thesis committee) can execute the scripts in 03_Code/ on the data in 02_Data/raw/ and perfectly regenerate the final datasets, models, and figures used in this thesis. This preserves the integrity and long-term scientific value of the entire research project.

3.2 DATA PREPROCESSING

The methodology and concepts discussed in the following sections are primarily based on established references in machine learning, particularly the books of Hossain (2023) and Raschka et al. (2022).

3.2.1 DATA CLEANING

Data cleaning plays an important role in preparing the sensor readings collected from the split-type air-conditioning units for modeling. Since the predictive maintenance model depends on accurate patterns in temperature, humidity, current draw, and voltage measurements, the raw dataset must first be checked for missing data, sudden spikes, duplicates. Any incomplete or duplicated entries are removed regardless of its severity. Outliers are either ignored or removed depending on its severity.

3.2.2 DATA SPLITTING

Holdout Method

Two splitting methods will be used and compared during the model evaluation. The **Holdout method** dataset into 3 parts. The training split that is used to train the model contains 60% of the dataset, the validation comprising of 20% of the dataset and is unseen to the model is used to understand the performance of the various models and parameters, the test split which is 20% of the dataset which is also unseen to the model is used to evaluate the model performance and determines its accuracy.

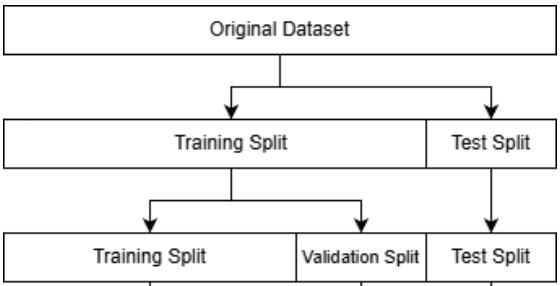


Fig 3.1 Holdout Method

k-Fold Cross Validation

Another method used is called k-fold Cross Validation where the training dataset is split into k-folds without replacement. The $k-1$ folds are used for the model training and the other fold called the test fold is used for performance evaluation like how validation split is used in the holdout method. This procedure is repeated k so that we obtain k models and performance estimates. We then calculate the average performance of the models based on the different, independent test folds to obtain a performance estimate that is less sensitive to the sub-partitioning of the training data compared to the holdout method. Typically, we use k-fold cross-validation for model tuning, that is, finding the optimal hyperparameter values that yield a satisfying generalization performance, which is estimated from evaluating the model performance on the test folds. Typically, we use k-fold cross-validation for model tuning, that is, finding the optimal hyperparameter values that yield a satisfying generalization performance, which is estimated from evaluating the model performance on the test folds

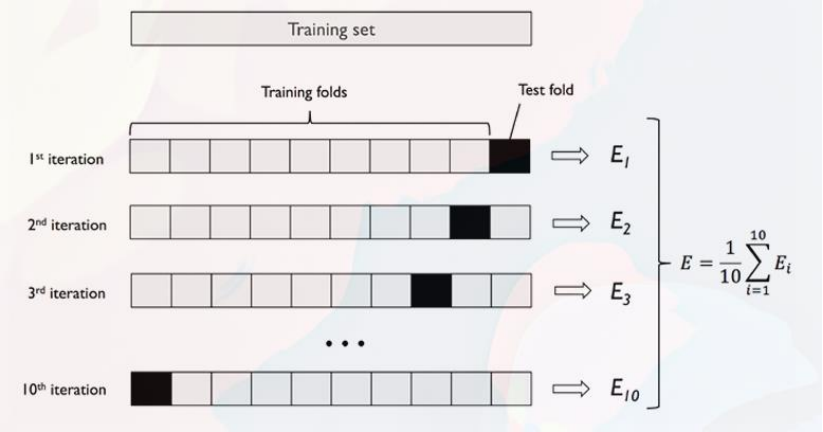


Fig 3.2 k-Fold Cross Validation Method Process

As for this research, the dataset is split first into 80-20 split using holdout method where 20% of the data is used as the testing set for final testing. While 80% of the data is used for k-fold cross validation with 5-folds.

k-folds	**Model 1** ***(accuracy)***	**…**	**Model n** ***(accuracy)***
1		…	
2		…	
3		…	
4		…	
5		…	

Fig 3.3 Sample Table of k-Fold CV

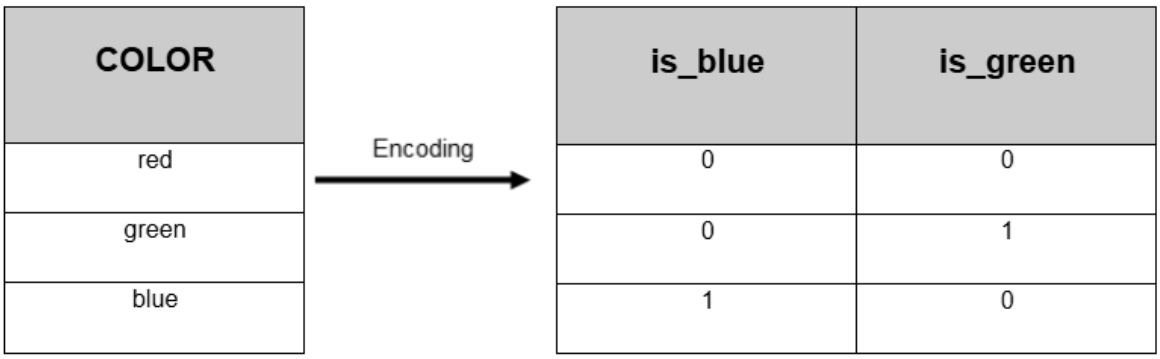
3.2.3 DATA TRANSFORMATION

Data Transformation serves as the next step after splitting dataset. Since the dataset contains different types of values such as continuous variables with different scaling such as current, temperature, and humidity and categorical variables such as frost build up and coil condition. The data needs to organized and transformed into forms that are more suitable for analysis and modeling. This process involves subprocesses such as *Data Encoding* and *Feature Scaling*.

Data Encoding

For this research that involves categorical variables, we will make use of **one-hot encoding** that transforms nominal variables into different columns containing binary variables called dummy variables. **One-hot encoding** is also called *dummy encoding*.

Fig 3.3 One-Hot Encoding on sample data



Feature Scaling

When working with different features or variables, it is almost certain that we have data on different features in different ranges which cannot be compared easily. Therefore, a sort of transformation is done on the attribute values so that all features come within a comparable and workable range. The transformation done on the data for this purpose is *feature scaling*. There are different methods for feature scaling which includes *standardization* that will be used for this research.

Standardization is a very popular method for feature scaling. After standardizing the dataset, we get a mean of zero and a unit standard deviation. The formula to perform standardization of the data is given in:

$$z = \frac{x - \mu}{\sigma}$$

where z represents the standardized value, x is the attribute value, μ is the respective attribute mean, and σ is the respective attribute standard deviation.

3.2.4 DATA REDUCTION

Datasets with too many features can cause issues like slow computation and overfitting. That is why this research is flawed for having too many features ($p = 11$) to be exact. Dimensionality reduction helps to reduce the number of features while retaining key information. It converts high-dimensional data into a lower-dimensional data while preserving important details. Dimensionality reduction techniques are divided into two categories namely *feature selection* and *feature extraction*. For this research *feature extraction* will be used.

Feature extraction involves creating new features by combining or transforming the original features. One technique under *feature extraction* that will be used is **Principal Component Analysis (PCA).**

Principal Component Analysis (PCA)

A feature extraction technique that converts correlated variables into uncorrelated principal components hence reducing dimensionality while maintaining as much variance as possible enabling more efficient analysis. **PCA** uses *linear algebra* to transform data into new features called principal components. Here are the following steps of **PCA**.

The training dataset is first standardized to make sure that each feature have the mean of 0 and a standard deviation of 1. This step has been already done in section 3.2.3 Data Transformation. Next, the *covariance matrix* is calculated to see how features relate to each other whether they increase or decrease together. A *covariance matrix* with dimensions of $m-$rows and $n-$columns has the following general formula:

$$C = \begin{bmatrix} \mathrm{cov}(x_1, x_1) & \cdots & \mathrm{cov}(x_n, x_1) \\ \vdots & \ddots & \vdots \\ \mathrm{cov}(x_1, x_n) & \cdots & \mathrm{cov}(x_n, x_n) \end{bmatrix}$$

Where n is the number of *features* and the formula for the $\mathrm{conv}(x, y)$ is:

$$\boldsymbol{cov(x_j, x_k) = \frac{1}{m-1} \sum_{i=1}^{m} (x_j - \bar{x}_j)(x_k - \bar{x}_k)}$$

where m is the number of rows/data.

The *eigenvalues,* λ of the covariance matrix C are calculated and ranked from highest to lowest. The *eigenvectors,* v of C represent the **principal components** whereas the corresponding *eigenvalues* will define their magnitude. Both the *eigenvalues* and *eigenvectors* are calculated using the following formula:

$$Cv = \lambda v$$

Or

$$\det(C - \lambda)v = 0$$

Since we want to reduce the dimensionality of our dataset by compressing it onto a new subspace, we only subset of *eigenvectors* or **principal components** that contain the most information (*variance)*. The eigenvalues define the magnitude of the eigenvectors, so we have to sort the eigenvalues by decreasing magnitude; we are interested in the top k eigenvectors based on the values of their corresponding eigenvalues. But before we collect those k most informative eigenvectors, a plot of the ratios of the variance is used to visualize the importance of the features. The variance explained ratio of an *eigenvalue*, λ_j, is simply the fraction of an eigenvalue, λ_j, and the total sum of the eigenvalues:

$$\text{Explained Variance Ratio} = \frac{\lambda_j}{\sum_{j=1}^{n} \lambda_j}$$

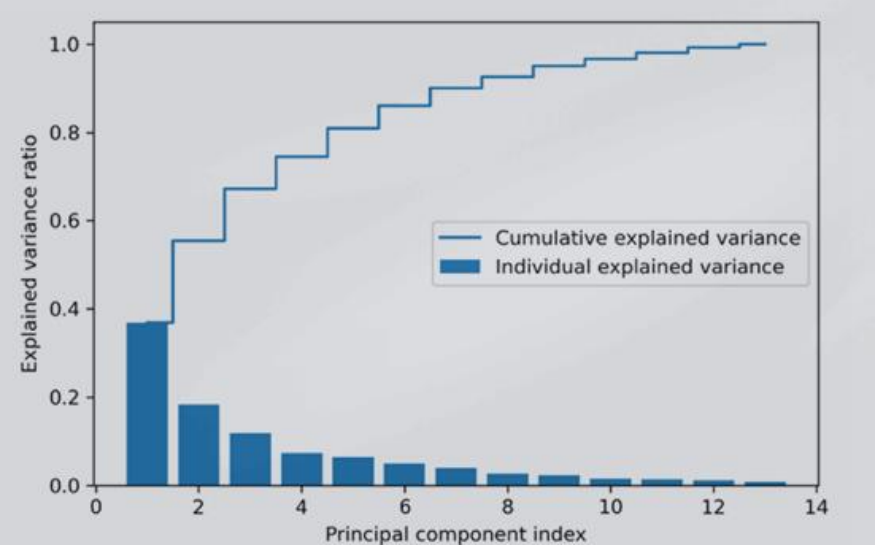


Fig 3.4 Bar Chart of the Principal Components

Select k eigenvectors, which correspond to the k largest eigenvalues, where k is the dimensionality of the new feature subspace ($k \leq n$) where the *cumulative explained variance* of k eigenvectors is at least 95%. A projection matrix W is constructed from the by horizontally stacking the *top k*

eigenvectors. The dataset X is then projected to the projection matrix W to get the transformed dataset X'.

$$\mathrm{X}' = \mathrm{XW}$$

Aside from transforming data, PCA can measure feature importance through the use of loadings. Loadings are the values located inside *an eigenvector or* ***principal component.*** Each row corresponds to one variable. **Loading percentage** is calculated through the following.

$$Ⅎr\{Feature\ Impor\tan ce$$

3.3 MODEL DEVELOPMENT

The first stage of the machine learning model involves classifying images into classes based on the feature that will be suitable for other machine learning models. The standard algorithm for computer vision (the inputs are image data) is CNN, a type of deep learning which is a subset of machine learning.

CNN is a type of feedforward neural network that learns features of images via filter optimization. Since CNN is a rather complex process compared to other machine learning and basic deep learning algorithms, it is better to understand the basic foundation of CNN though *ANN* (Artificial Neural Networks) or simply *Neural Networks* and *Deep Learning*

Neural Networks

Foundation: Deep Learning

Deep Learning is a large subset of *Machine Learning* that contains algorithms from all types of Machine Learning: *supervised, unsupervised,* and *reinforcement learning*. However, Deep Learning is built upon *Supervised Learning* where the perceptron, a special type of linear regression that used gradient descent, is its most basic form.

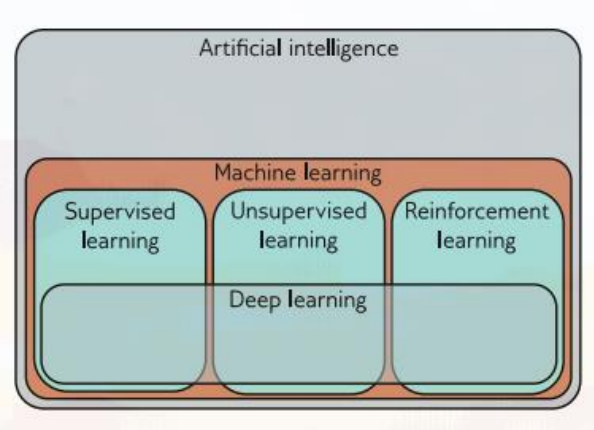


Fig 3.5 Artificial Intelligence, Machine Learning, and Deep learning

A *1D Linear Regression Model* describes the relationship between input x and output y as a straight line:

$$y = b + wx$$

This model has two parameters $\phi = [b, w]$ where b is the y-intercept of the line and w is the slope. Difference choices for the y-intercept and slope result in different relations between input and output. Different sets of these parameters can whether the model truly fits the input with the data leaving other models to be less or more accurate. That is why we need a principled approach for deciding which parameters ϕ are better than others. Hence, we assign a numerical value to each parameter that quantifies the degree of mismatch between the model and the data. We term this value the *loss*; a lower loss means a better fit. The mismatch captured by the deviation between the model predictions and the ground truth outputs. We quantify the total mismatch, *loss*, as the sum of the squares of these deviations for the I dataset.

$$L(\phi) = \sum_{i=1}^{n} [f(x_i, \phi) - y_i]^2$$

$$= \sum_{i=1}^{n} [b + wx - y_i]^2$$

The loss L is a function of the parameters ϕ; it will be larger when the model fit is poor, and smaller when it is good. We term $L(\phi)$ the *loss function* or *cost function*. The goal is to find the parameters ϕ that minimize the quantity:

$$\phi = \underset{\phi}{\arg min}[L(\phi)]$$

We can visualize the loss function as a surface. The best parameters are at the minimum of this surface. One method to find the best parameters is to use *gradient descent*.

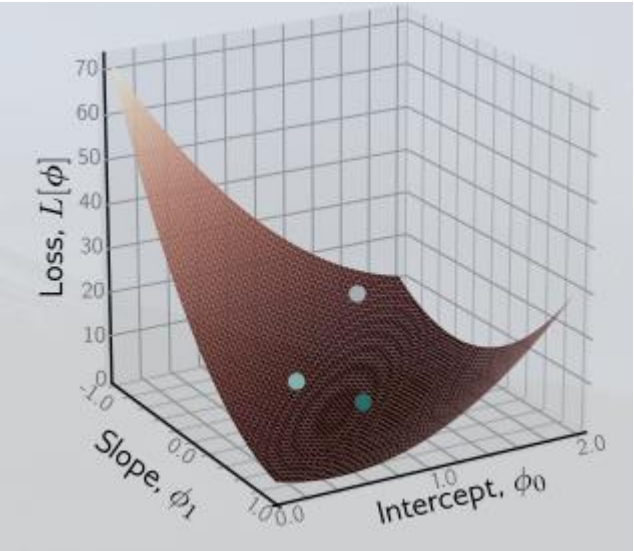


Fig 3.6 Graphing Loss Function with parameters

Shallow neural networks are functions $y = f(x, \phi)$ with parameters ϕ that map multivariate inputs x to multivariate outputs y. For this section, we will follow an example network $f(x, \phi)$ that maps scalar input x to a scalar output y and has ten parameters $\phi = \{\phi_0, \phi_1, \phi_2, \phi_3, \theta_{10}, \theta_{11}, \theta_{20}, \theta_{21}, \theta_{30}, \theta_{31}\}$. The sample network is shown below:

$$y = f(x, \phi)$$

$$y = \phi_0 + \phi_1 a(\theta_{11}x + \theta_{10}) + \phi_2 a(\theta_{21}x + \theta_{20}) + \phi_3 a(\theta_{31}x + \theta_{30})$$

The network is broken down into three parts where we first compute three linear functions an input of x $(\theta_{11}x + \theta_{10}, \theta_{21}x + \theta_{20}, \theta_{31}x + \theta_{30})$. We pass the 3 output to an *activation function* $a(z)$. There are many different types of activation function in deep learning but the most basic and common is *rectified linear unit* or *ReLU*:

$$a(z) = \text{ReLU}(z) = \begin{cases} 0, & z < 0 \\ z, & z \geq 0 \end{cases} = max(0, z)$$

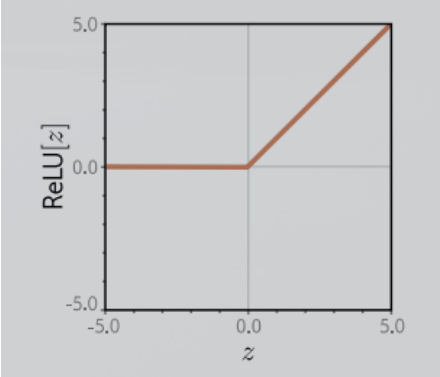


Fig 3.6 ReLU Graph

This returns the input when it is positive and zero otherwise. To make the function a bit more interpretable, we split the function into two parts by introducing intermediate quantities:

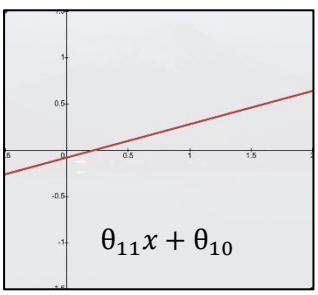
$$h_1 = a(\theta_{11}x + \theta_{10})$$

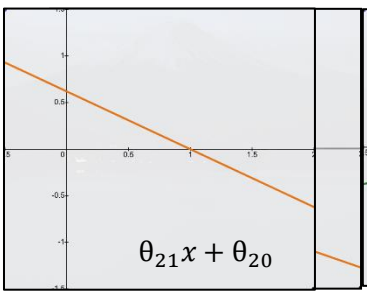
$$h_2 = a(\theta_{21}x + \theta_{20})$$

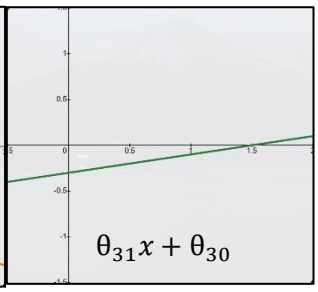
$$h_3 = a(\theta_{31}x + \theta_{30})$$

Where we refer to h_1, h_2, and h_3 as *hidden units* which we will replace in the neural network.

$$y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3$$







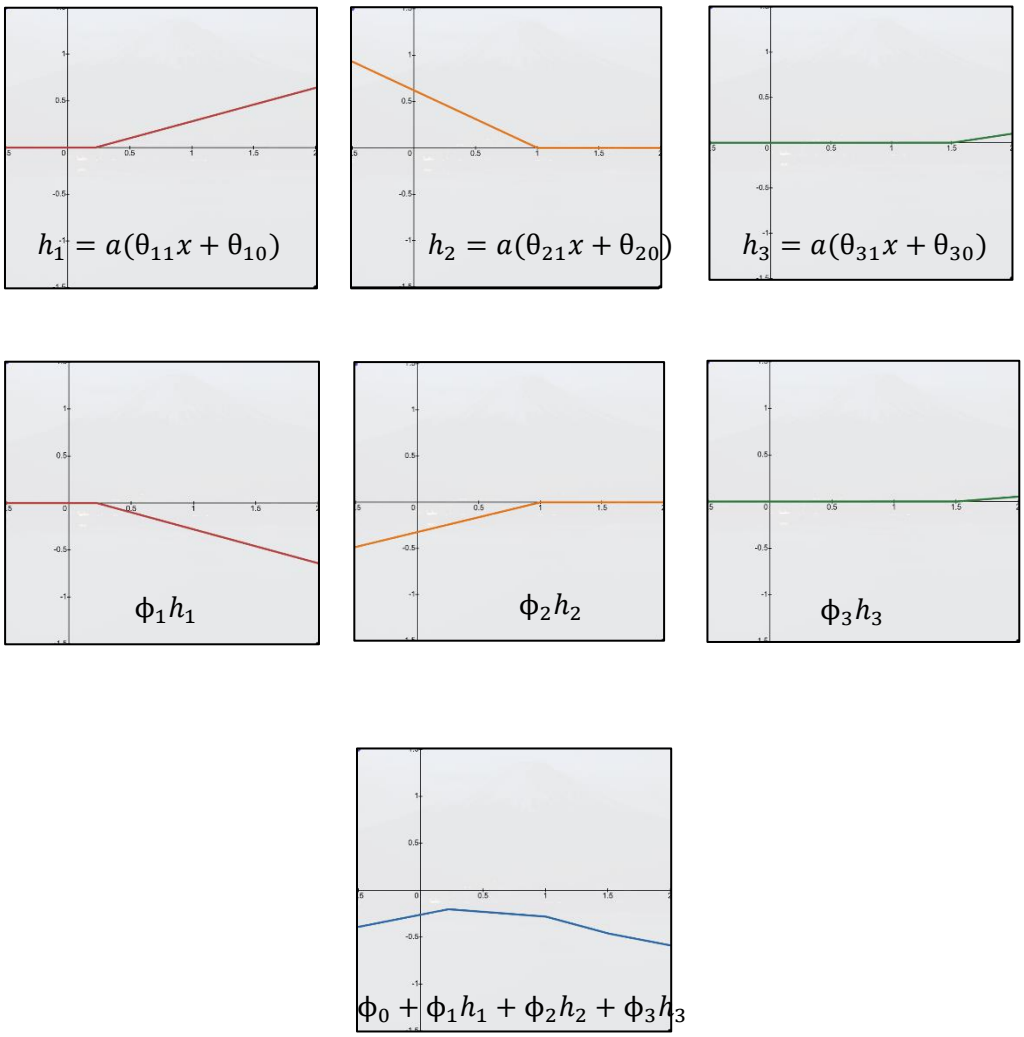


Fig 3.7 Graph of Neural Network and Steps

The neural network above using *hidden units* can be simplified and composed of three parts namely the input, output, and hidden layers. We can visualize a neural network through the following:

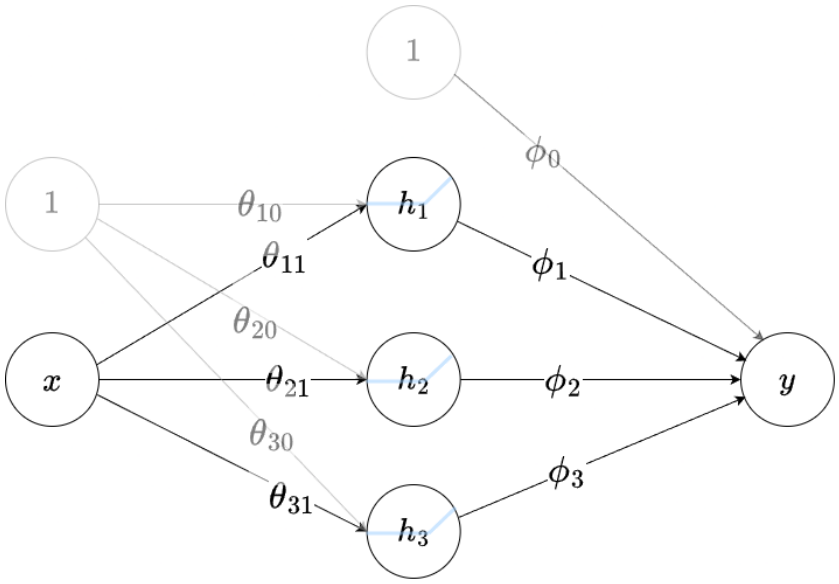


Fig 3.8 Graph of Neural Network and Steps

As shown in the figure above, neural networks such as this are often referred to in terms of layers. The left of the network is called the *input layer*, the center is the *hidden layer*, and to the right is the *output layer*. We could say for this network, the *hidden layer* contains 3 hidden units. These hidden units are commonly called as *neurons*.

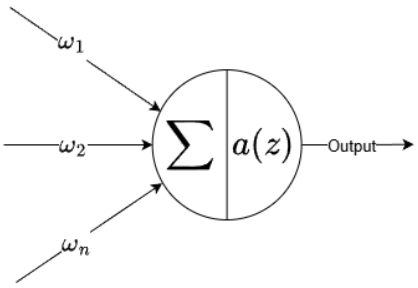


Fig 3.9 Visualization of a Neuron of a Perceptron Model

Aside from that, due to a large m number of data input, neural networks can be easily computed using linear algebra. The function of the neural network generally accepts matrices as inputs where x has a dimension of $m \times 1$ hence the function will have tweaks where the weights of one hidden layer containing k-number of *neurons* will have a dimension of $k \times 1$ and the biases will have a dimension of $m \times k$:

$$h = x\omega^T + b$$

$$h = \begin{bmatrix} x_1 \\ \vdots \\ x_m \end{bmatrix} [\omega_1 \quad \dots \quad \omega_k] + \begin{bmatrix} b_{11} & \dots & b_{1k} \\ \vdots & \ddots & \vdots \\ b_{m1} & \dots & b_{mk} \end{bmatrix}$$

Rather than having *1 input* and *1 outpu*t, a neural network can have multivariate inputs and outputs. Just like the network above, we can visualize networks with multivariate inputs and outputs with one hidden layer.

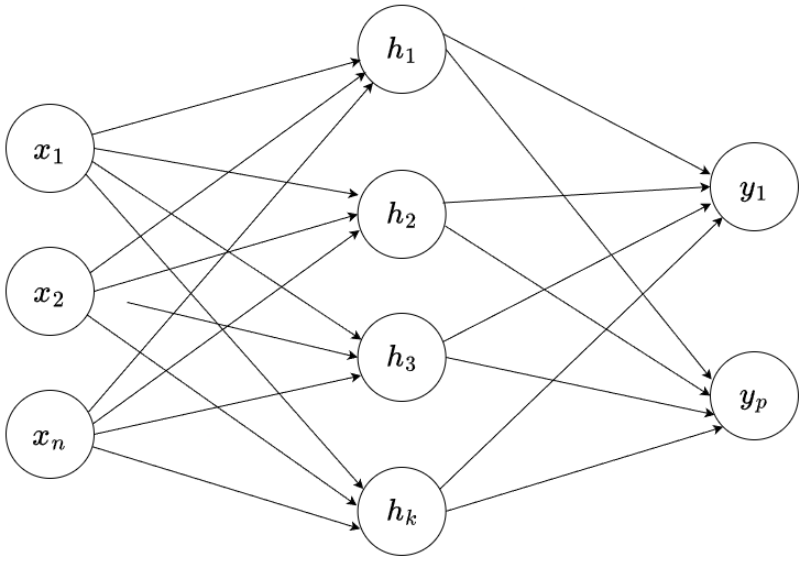


Fig 3.10 General Single Hidden Layer Network

A single hidden layer network can be generalized through different n number of inputs/features and p number of outputs. Therefore, the dimension of the input layer is $m \times n$ and the dimension of the output layer $p \times k$.

$$h = \begin{bmatrix} x_{11} & \cdots & x_{1n} \\ \vdots & \ddots & \vdots \\ x_{m1} & \cdots & x_{mn} \end{bmatrix} \begin{bmatrix} \omega_{11} & \cdots & \omega_{1k} \\ \vdots & \ddots & \vdots \\ \omega_{n1} & \cdots & \omega_{nk} \end{bmatrix} + \begin{bmatrix} b_{11} & \cdots & b_{1k} \\ \vdots & \ddots & \vdots \\ b_{m1} & \cdots & b_{mk} \end{bmatrix}$$

Deep Neural Network

*DNN*s are neural networks that contains more than one hidden layer which allows to have more descriptive power needed to describe datasets with high-dimensionality. This is because as hidden layers increase, there will be more linear regions that an approximate any function. Hence, if we want best results using deep neural networks are far better that neural networks that contains only one hidden layer (also called shallow networks). A visualization of a DNN is shown below:

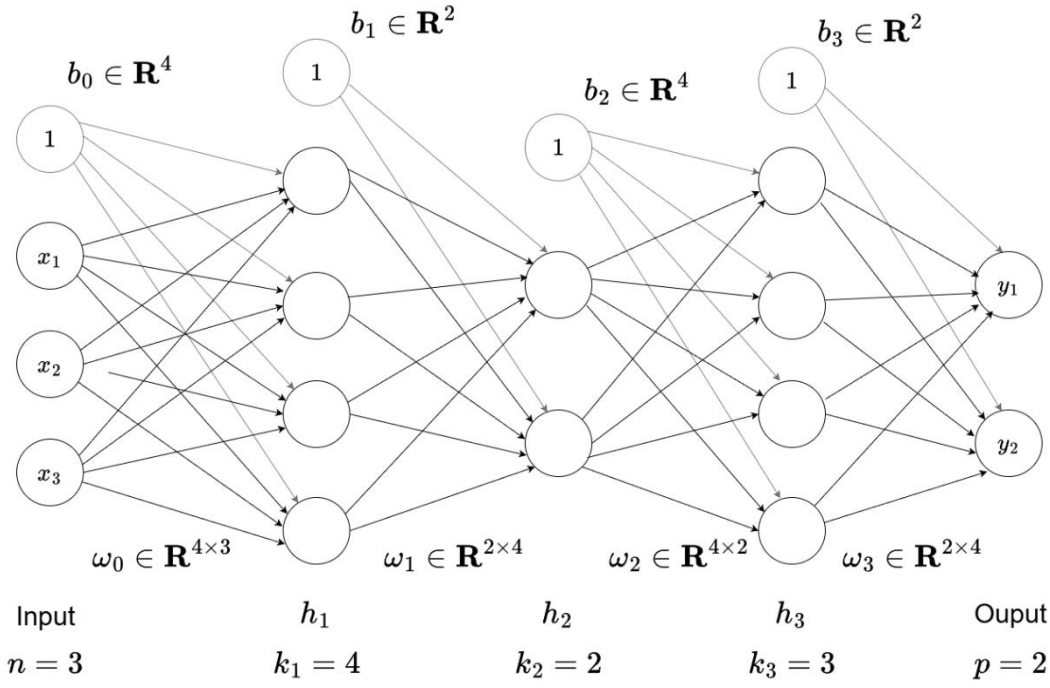


Fig 3.11 Deep Neural Network

In a deep neural network, more and more hidden layers can be stacked making it generalize complex functions at the expense of being more computationally expensive. That is why a formula of a general deep network $y = f(x, \phi)$ with K layers are written to avoid creating notations for each layers.

$$\boldsymbol{h_K = a(b_{K-1} + \omega_{K-1} h_{K-1})}$$

$$\boldsymbol{y = b_k + \omega_K h_K}$$

The parameters ϕ of this model comprise all of these weights matrices and bias vectors $\phi = \{\boldsymbol{b_k, \omega_k}\}_{k=0}^{K}$. For this research, we will be using a *DNN* model with a simple binary classification. A DNN with such task will have multi-variate inputs/features but will have one neuron output with a range of $[0,1]$. The network will use a *Binary Cross Entropy Loss* that uses *Bernoulli Distribution* and *negative log-likelihood* to quantify the difference between the actual binary labels and the predicted probabilities output by the model. The lower the binary cross-entropy value, the better the model's prediction aligns with the true labels.

Mathematically, **Binary Cross-Entropy** *(BCE)* is defined as:

$$L(\phi) = \sum_{\mathrm{i}=1}^{\mathrm{m}} -(1 - y_i)\, ln\big[1 - \mathrm{sig}[f(x_i, \phi)]\big] - y_m\, ln\big[\mathrm{sig}[f(x_i, \phi)]\big]$$

Where $sig[z]$ is the sigmoid function at input z that maps any real number into a value between 0 and 1:

$$sig[z] = \frac{1}{1 + e^{-z}}$$

To fit a model, we need a training set $\boldsymbol{X} = \{\boldsymbol{x}_i, \boldsymbol{y_i}\}$ of input/output pairs. We seek parameters ϕ for the model $f(x_i, \phi)$ that maps the inputs x_i to the outputs y_i as closely as possible through the use of optimization algorithms. The goal of an optimization algorithm is to find parameters ϕ that minimizes the loss:

$$\phi = arg\, \min_{\phi}[L(\phi)]$$

There are many families of optimization algorithms but the standard methods for training neural networks are iterative. The simplest method in this case is *gradient descent*. This starts with initial parameters (randomized) and iterate two steps:

1. Compute the partial derivatives of the loss with respect to the parameters:

$$\frac{\partial L}{\partial \phi} = \begin{bmatrix} \frac{\partial L}{\partial \phi_1} \\ \vdots \\ \frac{\partial L}{\partial \phi_N} \end{bmatrix}$$

2. Update the parameters according to the rule:

$$\phi \leftarrow \phi - \eta \cdot \frac{\partial L}{\partial \phi}$$

Where the positive scalar α determines the magnitude of the change.

The first step computes the gradient or slope of the loss function at the current position. The second step moves a small distance η *downhill* where we call it the *learning rate* which is one of the hyperparameter of a neural network. A hyperparameter is a parameter that is set before training a model and adjusted manually while a parameter ϕ is automatically adjusted during the learning process.

However, for this research, a stochastic gradient descent (SGD), a variant of gradient descent that offers several advantages in terms of efficiency and scalability. In traditional gradient descent, the gradients are computed based on the entire dataset which can be computationally expensive for large datasets while SGD instead is calculated on a small, randomized subset of the dataset.

Using SGD will help lessen the computational costs as traditional gradient descent is more computationally expensive since it requires us to update each weight based on the total N datapoints of the dataset. Consider a neural network $\boldsymbol{f}(\boldsymbol{x}, \boldsymbol{\phi})$ with multivariate input x, parameters ϕ, and three hidden layers h_1, h_2, and h_3:

$$\boldsymbol{h_1 = a(b_0 + \omega_0 x)}$$

$$\boldsymbol{h_2 = a(b_1 + \omega_1 h_1)}$$

$$\boldsymbol{h_3 = a(b_2 + \omega_2 h_2)}$$

$$\boldsymbol{f(x, \phi) = b_3 + \omega_3 h_3}$$

Where our individual loss term l_m, which return the negative log-likelihood of the ground truth label y_m, given the model prediction $\boldsymbol{f(x, \phi)}$ for the training input x_m. For this example, the loss is a *binary cross entropy loss* and the total loss is the sum of these terms over the training data:

$$l_m = -(1 - y_m)\, ln\big[1 - \text{sig}[f(x_i, \phi)]\big] - y_m\, ln\big[\text{sig}[f(x_i, \phi)]\big]$$

$$L(\phi) = \sum_{\text{i=1}}^{\text{m}} l_m$$

The most commonly used optimization algorithm for training networks is SGD, which updates the parameters as:

$$\phi_{t+1} = \phi_t - \eta \sum_{\text{i}\in \text{B}_\text{t}} \frac{\partial l_i(\phi_t)}{\partial \phi}$$

Where η is the *learning rate*, and B_t contains the batch indices at iteration t. Another algorithm that explains the training process fully is *Backpropagation*. This is an algorithm that combines both the *forward pass* (computing l_m) and *backward pass* (updating the weights). Based on the toy neural network above, the *forward pass* is treated as the computation of the loss as a series of calculations with randomized parameters ϕ:

$$h_1 = max(b_0 + \omega_0 x, 0)$$

$$h_2 = max(b_1 + \omega_1 h_1, 0)$$

$$h_3 = max(b_2 + \omega_2 h_2, 0)$$

$$y_1 = b_3 + \omega_3 h_3$$

$$f_1 = sig[y_1] = \frac{1}{1 + e^{-y_1}}$$

$$l_m = -(1 - y_m)\, ln[1 - f_1] - y_m\, ln[f_1]$$

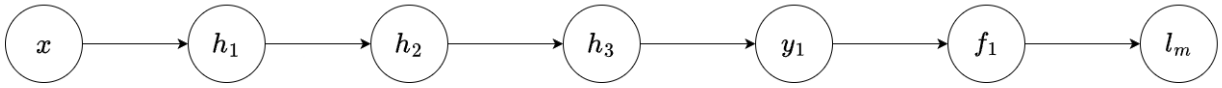


Fig 3.12 Forward Pass of a sample neural network

The *backwards pass* computes the derivatives of l_m with respect to the intermediate variables h_k, y, and f_1 in reverse order starting at the outermost hidden layer ($K = 3$). For example, the derivatives of l_m with respect to f_1 and y_1 are:

$$\boldsymbol{\frac{\partial l_m}{\partial f_1} = \frac{f_1 - y_m}{f_1(1 - f_1)}}$$

$$\frac{\partial l_m}{\partial y_1} = \frac{\partial l_m}{\partial f_1} \cdot \frac{\partial f_1}{\partial y_1} = \frac{f_1 - y_m}{f_1(1 - f_1)} \cdot [f_1(1 - f_1)]$$

$$\boldsymbol{\frac{\partial l_m}{\partial y_1} = f_1 - y_m}$$

We can also calculate the derivatives for each of the *intercept* b and the *slope* ω of each layer. For example, the derivatives of l_m with respect to slope and intercept of y_1 and h_3 are:

$$\frac{\partial l_m}{\partial b_3} = \frac{\partial l_m}{\partial f_1} \cdot \frac{\partial f_1}{\partial y_1} \cdot \frac{\partial y_1}{\partial b_3} = (f_1 - y_m) \cdot 1$$

$$\frac{\partial l_m}{\partial b_3} = f_1 - y_m$$

$$\frac{\partial l_m}{\partial \omega_3} = \frac{\partial l_m}{\partial f_1} \cdot \frac{\partial f_1}{\partial y_1} \cdot \frac{\partial y_1}{\partial \omega_3} = (f_1 - y_m) \cdot h_3$$

$$\frac{\partial l_m}{\partial \omega_3} = h_3(f_1 - y_m)$$

$$\frac{\partial l_m}{\partial b_3} = \frac{\partial l_m}{\partial f_1} \cdot \frac{\partial f_1}{\partial y_1} \cdot \frac{\partial y_1}{\partial h_3} \cdot \frac{\partial h_3}{\partial b_2} = (f_1 - y_m) \cdot \omega_3 \cdot 1$$

$$\frac{\partial l_m}{\partial b_2} = \begin{cases} \omega_3(f_1 - y_m) & if(b_2 + \omega_2 h_2 > 0) \\ 0 & if(b_2 + \omega_2 h_2 \leq 0) \end{cases}$$

$$\frac{\partial l_m}{\partial b_3} = \frac{\partial l_m}{\partial f_1} \cdot \frac{\partial f_1}{\partial y_1} \cdot \frac{\partial y_1}{\partial h_3} \cdot \frac{\partial h_3}{\partial \omega_2} = (f_1 - y_m) \cdot \omega_3 \cdot h_2$$

$$\frac{\partial l_m}{\partial \omega_2} = \begin{cases} \omega_3 h_2(f_1 - y_m) & if(b_2 + \omega_2 h_2 > 0) \\ 0 & if(b_2 + \omega_2 h_2 \leq 0) \end{cases}$$

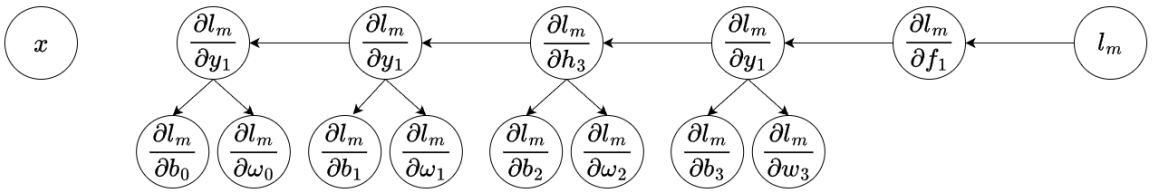


Fig 3.13 Backward Pass of a sample neural

During training, the model processes the dataset multiple times in a cycle. Each complete pass through the entire dataset is called an epoch. The number of epochs is a key hyperparameter; by

repeating the process of *backpropagation* over several epochs, the model can iteratively learn and improve its accuracy.

Developing *DNN*s involves different processes such as developing its architecture and using backpropagation to train the model. A flowchart is shown below to encapsulate the algorithm of developing a *DNN*.

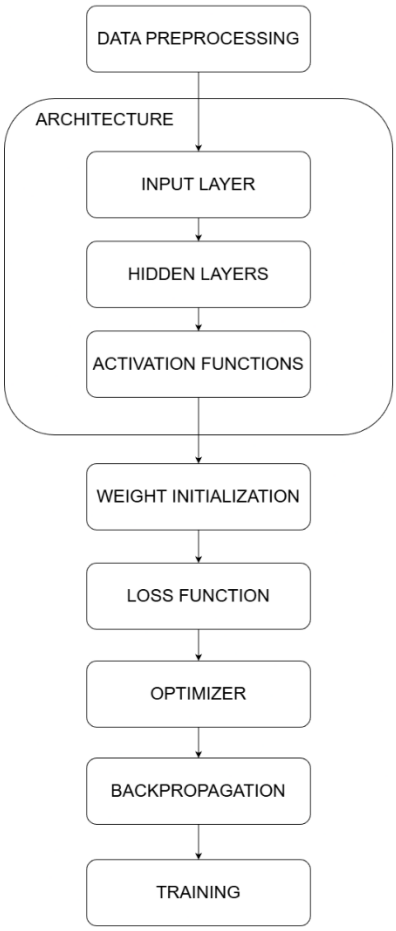


Fig 3.14 DNN Development Flowchart

Convolutional Neural Networks

CNNs are neural networks that have special layers called *convolutional layers*. The problem with using shallow or deep neural networks for image classification is that the number of weights would be extremely huge considering that each RGB value of an image has 150,528 input dimensions. Through the use of convolutional layers, each region of an image is independently processed using parameters shared across the whole image. Hence, it would use fewer parameters than fully connected layers.

Convolutional layers are network layers based on the *convolution* operation. In 1D, a convolution transforms an input vector x into an output vector z so that each output z_i is a weighted sum of nearby inputs. The same weights are used at every position and are collectively called the *convolution kernel* or *filter*. The size of the region over which inputs are combined is termed the *kernel size*. For a kernel size of three we have:

$$z_i = \omega_1 x_{i-1} + \omega_2 x_i + \omega_3 x_{i+1}$$

where $\boldsymbol{\omega} = [\omega_1, \omega_2, \omega_3]^{\mathbf{T}}$ is the kernel.

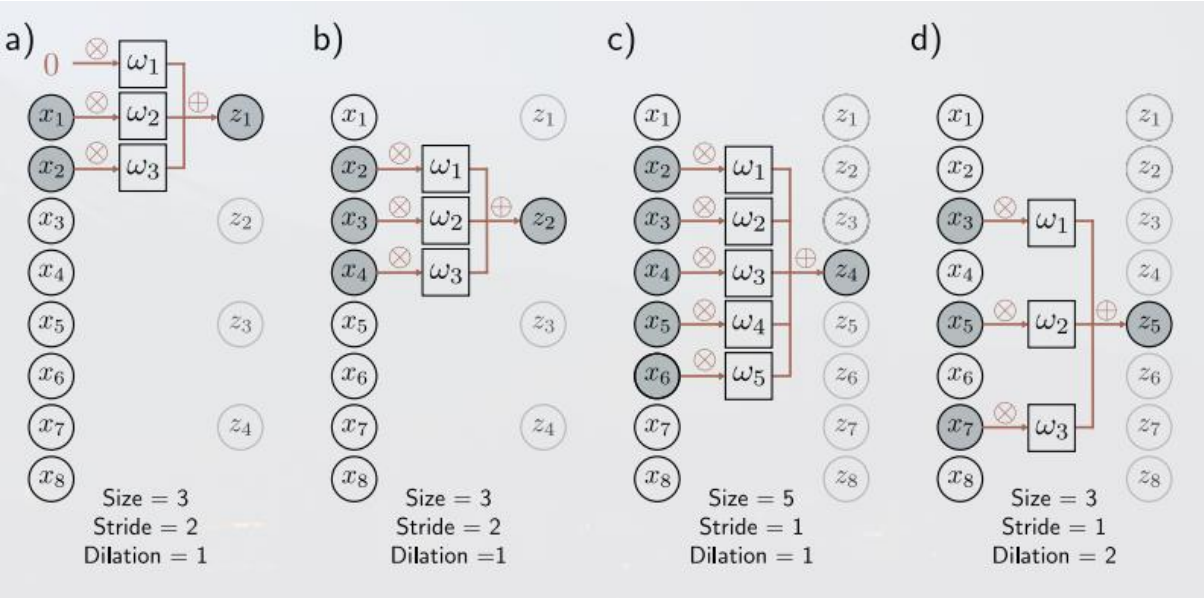


Fig 3.15 Convolutional Layers

Convolutional layers are distinguished by 3 characteristics namely *padding* where the input is padded with values of 0 or mirrored values, *stride* lets us skip every other input or by how many inputs, *kernel size* lets us increase the region, it typically remains an odd number so that it can be centered around the current position. A convolution layer then computes its output by convolving the input, adding a bias b, and passing each result through an activation function $a[z]$.

$$h_i = a\left[b_i + \sum_{j=1}^{D} \omega_{ij}\, x_j\right]$$

If there are *D* inputs x and *D* hidden units h, this fully connected layer would have D^2 weights ω and *D* biases b. If we only apply a single convolution, information will likely be lost. Hence, it is usual to compute several convolutions in parallel. Each convolution produces a new set of hidden variables, termed a *feature map* or *channel*.

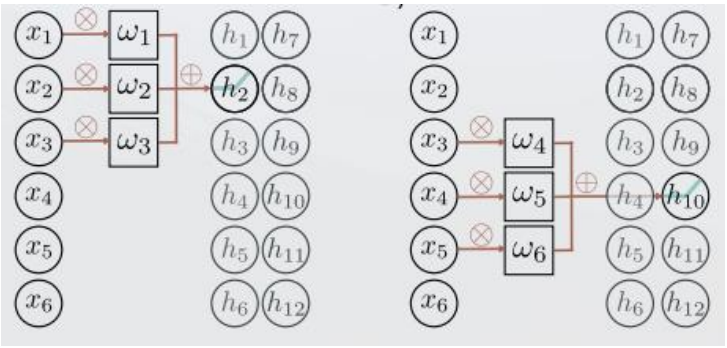


Fig 3.16 Convolutional Layers with 2 output channels

The first kernel in the figure computes a weighted sum of the nearest three pixels, adds a bias, and passes the results through the activation function to produce hidden units h_1 to h_6. These comprise the first channel. The second kernel computes a different weighted sum of the nearest three pixels, adds a different bias, and passes the results through the activation function to create hidden units h_7 to h_{12}. These comprise the second channel.

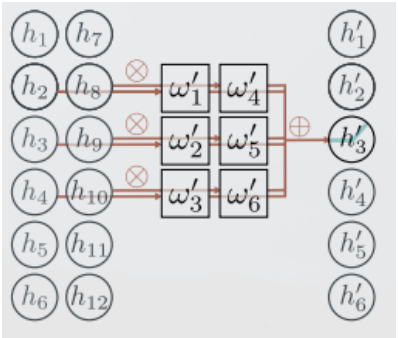


Fig 3.17 Multi-Input Channels

In general, the input and the hidden layers all have multiple channels. If the incoming layer has C_i channels and we select a kernel size K per channel, the hidden units in each output channel are computed as a weighted sum over all C_i channels and K kernel entries using a weight matrix $\boldsymbol{\omega} \in \boldsymbol{R}^{\boldsymbol{C_i} \times \boldsymbol{K}}$ and one bias. Hence, if there are C_o channels in the next layer, then we need $\boldsymbol{\omega} \in \mathbf{R}^{\boldsymbol{C_i} \times \mathbf{C_o} \times \boldsymbol{K}}$ weights and $\mathbf{b} \in \mathbf{R}^{\mathbf{C_o}}$ biases.

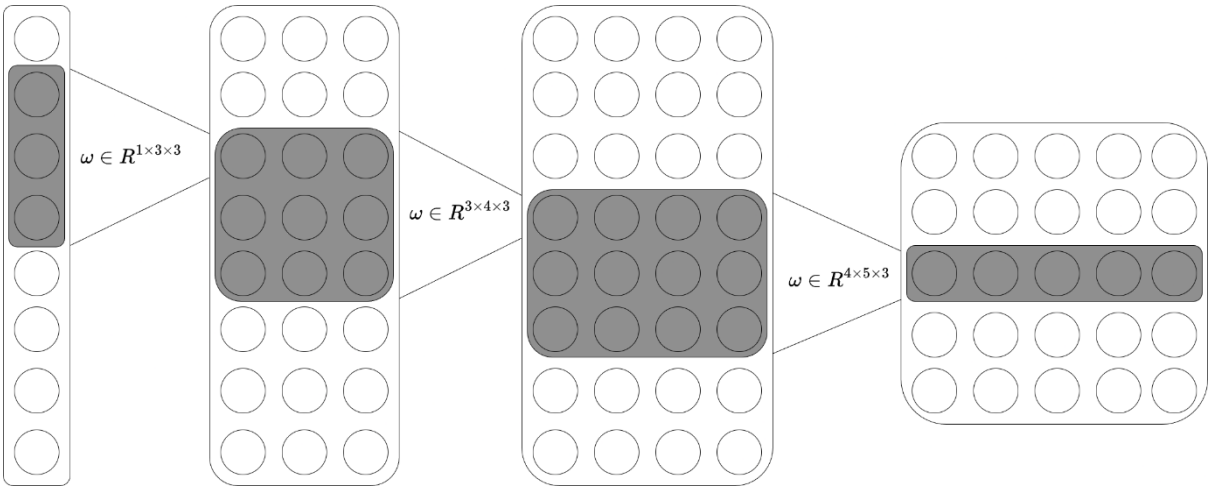


Fig 3.17 Stacked Convolutional Layers

.

However, convolutional networks are more usually applied to 2D image data. The convolutional kernel then is now a 2D object. A 3×3 kernel $\boldsymbol{\omega} \in \boldsymbol{R}^{\mathbf{3\times3}}$ applied to a 2D input comprising of elements x_{ij} computes a single layer of hidden units h_{ij} as:

$$h_{ij} = a\left[b + \sum_{m=1}^{3}\sum_{n=1}^{3} \omega_{mn} x_{i+m-2,j+n-2}\right]$$

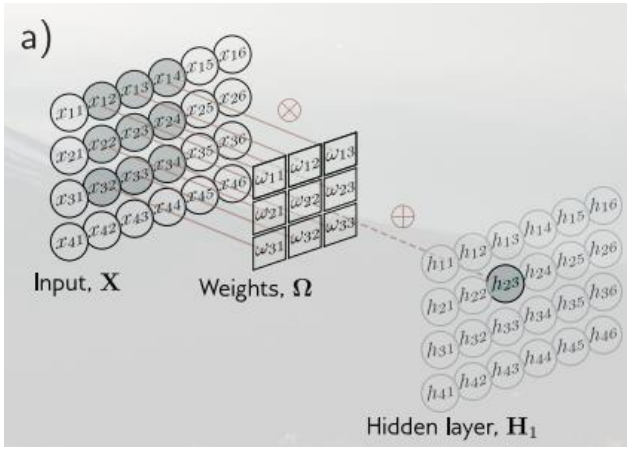


Fig 3.18 Stacked Convolutional Layers

Often the input is an RGB image, which is treated as 2D signal with three channels. A 3×3 kernel would have $3 \times 3 \times 3$ weights and be applied to the three input channels at each 3×3 positions. If the kernel size is $K \times K$, and there are $\mathrm{C_i}$ input channels, each output channel is a weighted sum of $C_i \times K \times K$ quantities plus one bias. It follows that to compute C_o output channels, we need $C_i \times \mathrm{C_o} \times K \times K$ weights and C_o biases.

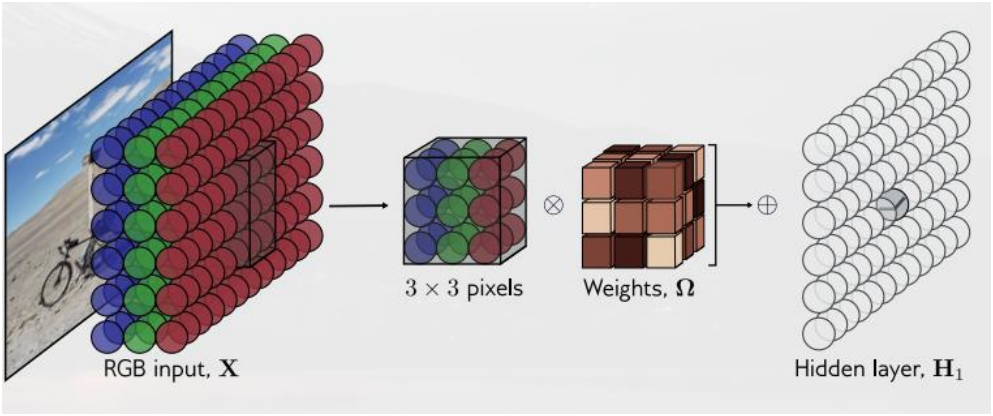
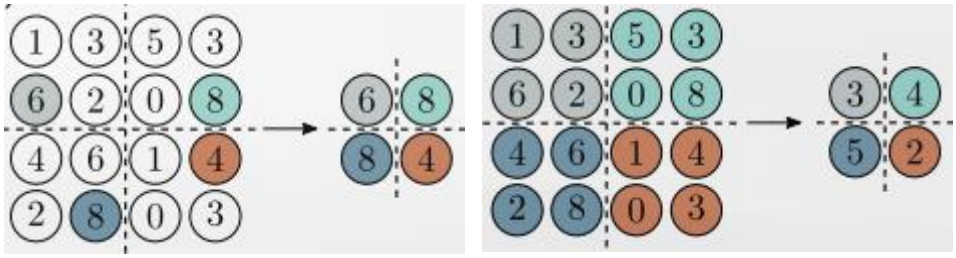


Fig 3.19 2D Convolution applied to an image

Downsampling scales down each feature map which minimizes unwanted features of the image and selecting the most important features of the image. Thereby, decrease the computation load and allowing to have more channels in the next hidden layer which allows us to extract more features. *Pooling* is layer that allows for *downsampling* by retaining the maximum of the 2×2 input values (*Max Pooling*) or taking the average of the inputs (*Mean Pooling*).



Changing the number of channels as previously mentioned allows us to learn more complex features of an image. These features are extracted on each channel of the hidden layers. Features of an image can be categorized as either low-level or high-level. Low-level features include *contours, edges, edges, angles,* and *colors.* High-Level Features are generated from low-level feature pairings and contains more complicated details about an image's or video's topic. This includes *items, faces, shapes*, and *interactions*.

CNN is hugely applied in computer vision which allows for image classification and object detection. Much of the pioneering work of CNN focused on classifying images from the ImageNet dataset. This contains 1,281,167 training images, 50,000 validation images, and 100,000 test images, and every image is labeled as belonging to one of 1000 possible categories. In 2012, *AlexNet* was the first convolutional network to perform well on this task. It consists of eight hidden layers with ReLU activation functions, of which the first five are convolutional and the rest fully connected.

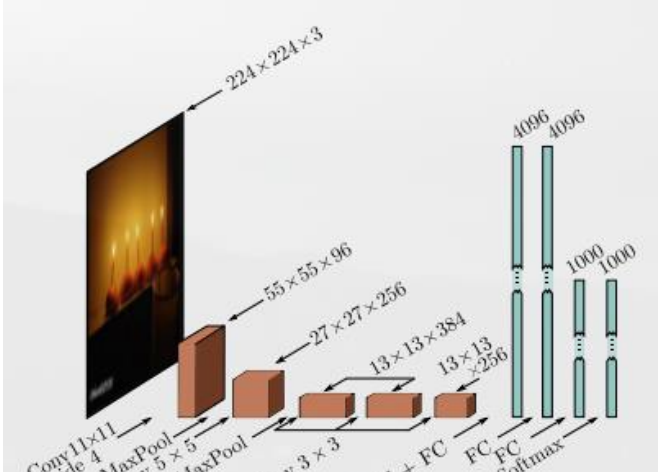


Fig 3.21 AlexNet (Krizhevsky et al., 2012)

k-Nearest Neighbors

Among the models that will be used, *k-NN classifier is* the easiest model to work with. KNN is a typical example of a **lazy learner**. It is called “lazy” not because of its apparent simplicity but because it doesn’t learn a discriminative function like neural networks from the training data but the memorizes the training dataset instead.

The KNN algorithm itself is fairly straightforward and can be summarized by the following steps:

1. Choos the number of k and a distance metric
2. Find the k-nearest neighbors of the data record that we want to classify
3. Assign the class label majority vote.

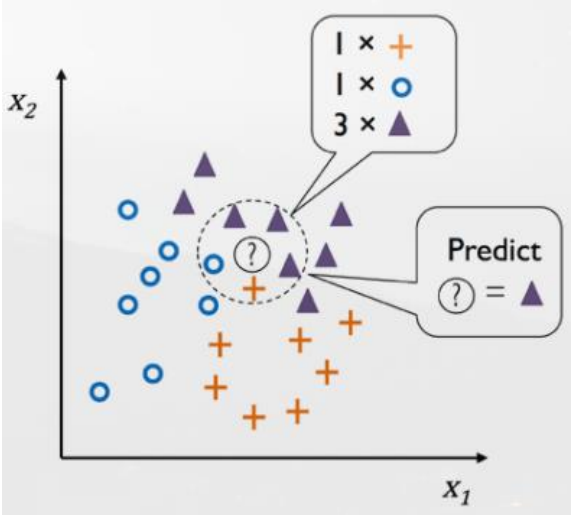


Fig 3.22 Majority Voting with 5 nearest neighbors

The *right* choice of k is crucial to finding a good balance between overfitting and underfitting. We also have to make sure that we choose a distance metric that is appropriate for the features in the dataset. One example of a distance metric is the *Minkowski Distance* which is just a generalization of the two distance metrics namely *Euclidean* and *Manhattan* distance.

$$d\left(x^{(i)}, x^{(j)}\right) = \sqrt[p]{\sum_k \left|x_k^{(i)} - x_k^{(j)}\right|^p}$$

It becomes the Euclidean distance if we set the parameter p=2 or the Manhattan distance at p=1. However, it is important to mention that KNN is very susceptible to overfitting due to the *curse of dimensionality* where the model fails to generalize the high number of features.

Decision Tree

Decision Tree classifiers are attractive models if we care about *interpretability*. As the name suggests, we can think of these models as breaking down our data by making a decision based on asking a series of yes or no questions.

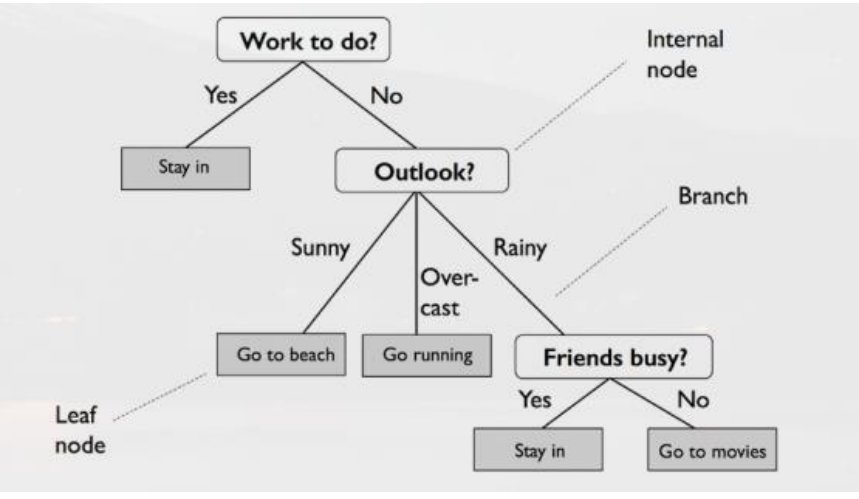


Fig 3.23 An example of a decision tree

Using the decision algorithm, we start at the tree root and split the data on the feature that results in the largest *information gain* IG. In an iterative process, we can then repeat this splitting procedure at each child node until the leaves are pure. This means that the training examples at each node all belong to the same class. In practice, this can result in a very deep tree with many nodes, which can easily lead to overfitting. Thus, we typically want to prune the tree by setting a limit for the maximum depth of the tree.

To split the nodes at the most informative features, we need to define an objective function to optimize via the tree learning algorithm. Here, our objective function is to maximize the IG at each split, which we define as follows:

$$\boldsymbol{IG(X_p, f) = I(X_p) - \sum_{j=1}^{m} \frac{N_j}{N_p} I(X_j)}$$

Where, f is the feature to perform the split; X_p and X_j are the dataset of the parent and jth child node; I is impurity measure; N_p is the total number of training examples at the parent node; and N_j is the number of examples in the jth child node. However, for simplicity and to reduce the combinatorial search space, most libraries (including scikit-learn) implement binary decision trees. This means that each parent node is split into two child nodes, X_{left} and X_{right}:

$$\boldsymbol{IG(X_p, f) = I(X_p) - \frac{N_{left}}{N_p} I(X_{left}) - \frac{N_{right}}{N_p} I(X_{right})}$$

The three impurity measures or splitting criteria that are commonly used in binary decision trees are **Gini Impurity** (I_G), **entropy** (I_H), and the **classification error** (I_E) two of which (*gini* and *entropy*) are used for this research. The mathematical representation for entropy where c is the total number of classes/labels:

$$\boldsymbol{I_H(t) = -\sum_{i=1}^{c} p(i|t)\, log_2\, p(i|t)}$$

The *gini impurity* can be understood as a criterion to minimize the probability of misclassification:

$$\boldsymbol{I_G(t) = 1 - \sum_{i=1}^{c} p(i|t)^2}$$

The classification error however can be used to *prune* the decision tree which helps from overfitting the data.

$$\boldsymbol{I_E(t) = 1 - \max p(i|t)}$$

There are ways to prevent overfitting and one of the basic methods (these methods are implemented during training) include:

- **Maximum Depth**: Limits the depth of the tree
- **Minimum Samples per Leaf**: Ensuring that each leaf node contains a minimum number of samples
- **Minimum Samples per Split**: Specifying the minimum number of samples required to perform a split.
- **Minimum Number of Leaf Nodes**: Controlling the number of leaf nodes in the tree.
- **Impurity Threshold:** Stopping when the impurity (Gini impurity or entropy) falls below a certain threshold.

Ensemble Methods

Ensemble methods have gained huge popularity in applications of machine learning during the last decade due to their good classification performance and robustness toward overfitting. These are methods where we use small models instead of just one. Each of these models may not be very strong on its own, but when we put their results together, we get a better and more accurate answer. There are three main types of ensemble methods (two of which will be used for this research):

1. *Bagging*: Models are trained independently on different random subsets of the training data. Their results are then combined-usually by averaging (for regression) or voting (for classification).
2. *Boosting:* Models are trained one after another. Each new model focuses on fixing the errors made by the previous ones. The final prediction is a weighted combination of all models.

Random Forest

RF is a type of bagging method where its algorithm can be summarized in four steps:

1. Draw a random **bootstrap** sample of size n (randomly choose n examples from the training dataset with replacement).
2. Grow a decision tree from the bootstrap sample. At each node:
    a. Randomly select d features without replacement.
    b. Split the node using the feature that provides the best split according to the objective function, for instance, maximizing the information gain.
3. Repeat steps 1-2 k times where k is the number of trees.
4. Aggregate the prediction by each tree to assign the class label by *majority vote*.

Although random forests don't offer the same level of interpretability as decision trees, a big advantage of random forests is that we don't have to worry so much about choosing good hyperparameter values. We typically don't need to prune the random forest since the ensemble model is quite robust to noise from averaging the predictions among the individual decision trees. The only parameter that we need to care about in practice is the number of trees, k, (step 3) that we choose

for the random forest. Typically, the larger the number of trees, the better the performance of the random forest classifier at the expense of an increased computational cost.

Fig 3.24 An example of a random forest

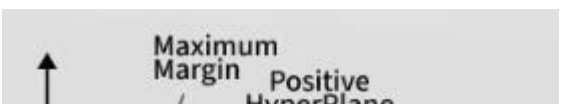
XGBoosting

XGBoosting or also known as eXtreme Gradient boosing is an optimized implementation of *Gradient Boosting* which is a type of ensemble learning method. It builds decision trees sequentially with each tree attempting to correct the mistakes made by the previous one. The process can be broken down as follows:

1. Start with a base learner: The first model decision tree is trained on the data.
2. Calculate the error using Loss Functions as mentioned in the Neural Networks Section
3. Train the next tree.
4. Repeat the process.
5. Combine the predictions.

Support Vector Machines

Support Vector Machine is a supervised machine learning algorithm used for classification and regression tasks. It tries to find the best boundary known as hyperplane that separates different



classes in the data. The main goal of SVM is to maximize the margin between the two classes. The larger the margin the better the model performs on new and unseen data.

Fig 3.25 Support Vector Machine Terms

Support Vectors are data points that lie closest to the decision surface (or hyperplane). The key idea behind the SVM algorithm is to find the hyperplane that best separates two classes by maximizing the margin between them. This margin is the distance from the hyperplane to the nearest data points (support vectors) on each side. The best hyperplane also known as the **"hard margin"** is the one that maximizes the distance between the hyperplane and the nearest data points from both classes.

Consider a binary classification problem with two classes, labeled as +1 and -1. We have a training dataset consisting of input feature vectors X and their corresponding class labels Y. The equation for the linear hyperplane can be written as:

$$\omega^T x_i + b = 0$$

The distance between a data point $x_i$and the decision boundary can be calculated as where $||\omega||$ is the unit vector of the weight vector:

$$d_i = \frac{\omega^T x_i + b}{||\omega||}$$

In a linear SVM Classifier, the data can be classified based on the distance from the data to the hyperplane:

$$\hat{y} = \begin{cases} 1 & : \omega^T x_i + b \geq 0 \\ -1 & : \omega^T x_i + b \geq 0 \end{cases}$$

For a linearly separable dataset the goal is to find the hyperplane that maximizes the margin between the two classes while ensuring that all data points are correctly classified. This leads to the following optimization problem:

$$\text{minimize}_{w,b} \frac{1}{2} ||\omega||^2$$

Hyperparameter Tuning

An ML model consists of some hyperparameters and parameters. The job of an ML model is to learn the parameters to give the correct hypothesis in a given context, but the structure of the ML model depends on the hyperparameters. It is evident that an optimized ML model is expected for the final application, so a proper choice. The steps associated with hyperparameter tuning are given below:

1. Visualize the data and understand the problem.

2. Select the best possible ML algorithm suitable for that problem.

3. Split the dataset into three sets—train set, validation set, and test set.

4. Determine the list of parameters and create the hyperparameter space (HS).

5. Select the most suitable method for searching the optimal set of hyperparameters from the HS and apply that.

6. Implement cross-validation.

7. Evaluate the model score.

8. Repeat steps 5, 6, and 7 until the best possible model score is achieved. The hyperparameter set with the best model score is expected to be optimal.

3.4 MODEL EVALUATION

Confusion Matrix

All machine learning models, especially for classification tasks can be evaluated using a confusion matrix. A confusion matrix gives a comprehensible understanding of the performance of a given classification model. Whether an evaluation score is meaningful or not can be understood using a confusion matrix. The terms used in the confusion matrix in are described below:

1. *True Positive (TP):* These are the positive cases, and the model correctly predicted them as positive cases.
2. *False Positive (FP):* These are the cases that are not positive, but the model predicted them as positive. This error is a type 1 error. 3.
3. *True Negative (TN)*: These are the negative cases, and the model correctly predicted them as negative cases. 4.
4. *False Negative (FN)*: These are the cases that are actually positive, but the model has incorrectly predicted them as negative cases. This error is a type 2 error.

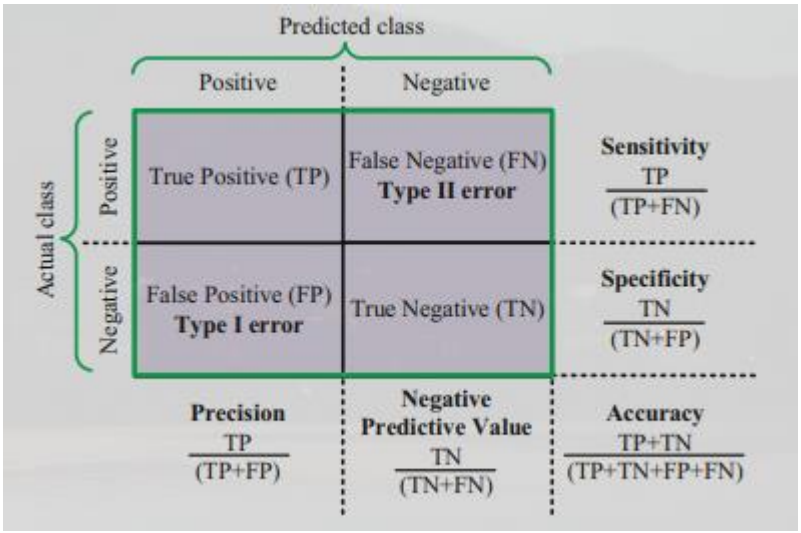


Fig 3.26 Confusion Matrix

Accuracy

It is the simplest form of evaluation score. It is defined by the number of correctly predicted observations over the total number of observations. The accuracy score is valid when the dataset is balanced; each class in the dataset has an equal number of data objects. However, for an imbalanced dataset, the accuracy score can be meaningless as it does not provide a detailed insight into the model's performance

$$Accuracy = \frac{TP + TN}{TP + FP + FN + TN}$$

Precision and Recall

The precision score is used to understand the ratio of correctly predicted positive cases among all the positive cases predicted by the model. It is used when the system is required to have a low false-positive rate.

$$Precision = \frac{TP}{TP + FP}$$

The recall score, also called sensitivity, is used to calculate the true-positive rate. It is calculated as the ratio of correctly predicted positive cases to all the cases in the actual positive class. It is used when the system is required to have a low false negative rate.

$$Recall = \frac{TP}{TP + FN}$$

There is a general trade-off between precision and recall. A system cannot simultaneously have a high precision score and a high recall score. A high precision leads to a poor recall score and vice versa.

F1 Score

The F1 score is the harmonic mean of precision and recall score. It is used when both the precision and recall scores need to be considered for the system.

Statistical Analysis

After the dataset are cross validated using k-fold cross validation during the training process. The test split is grouped into 30 subsets which are cross validated across *n* models. Each model are validated in terms of *accuracy, precision, recall,* and *F1 Score*.

Subset	**Model 1** ***(accuracy)***	**…**	**Model n** ***(accuracy)***
1		…	
2		…	
3		…	
…		…	
30		…	

Fig 3.26 Accuracy Table

Since the dataset is non-normal due to having a standard accuracy range of *80%-95%*. A *Friedman Test* will be used.

Freidman Test

The **Friedman Test** is a non-parametric alternative to the Repeated Measures It is used to determine whether or not there is a statistically significant difference between the means of three or more groups in which the same subjects show up in each group. The hypothesis of this research for this statistical analysis is the following.

H_0: *The means across the models are all equal.*

H_1: *At least one population means is different from the rest.*

3.5 MODEL DEPLOYMENT

8. Model Deployment and Application

The final phase of this methodology is the deployment of the selected predictive model. The objective of deployment is to transition the trained model from a static, saved file (e.g., a .pkl or .pth file stored in the 04_Outputs/models/ directory) into a functional, active system. This system will serve as a decision-support tool, providing actionable insights to researchers and maintenance personnel by displaying the real-time health status of the monitored air conditioning units.

Given this project's context as a university-based research study, a pragmatic and achievable deployment strategy is prioritized. The goal is to create a functional prototype that demonstrates the model's value without requiring complex, enterprise-level web development infrastructure.

8.1. Deployment Architecture and Technology Selection

The model will be deployed as a local web application dashboard hosted on the central "Ingest Server". This server is already designated to receive data from all 50 "Master" ESP32 nodes, making it the logical location to also host the "brain" of the system.

This architecture is chosen for its simplicity and efficiency, requiring only Python-native libraries that are well-suited for a mechanical engineering student with an average coding background.

Application Framework: Streamlit The user-facing dashboard will be built using Streamlit, an open-source Python framework. This technology is explicitly chosen because it allows data scientists and engineers to create interactive, data-driven web apps using only Python scripts, requiring no front-end experience in HTML, CSS, or JavaScript. This dramatically lowers the barrier to creating a functional and professional-looking dashboard for displaying model predictions and sensor data.

Prediction Database: SQLite To store the model's predictions over time, a local SQLite database will be used. SQLite is a serverless, self-contained database engine built directly into Python via the sqlite3 module. It stores the entire database in a single file on the server.

This solution is ideal for a research prototype as it provides robust data storage for predictions without the installation and management overhead of a full-scale database server.

Automation: Watchdog To automate the prediction pipeline, the Python watchdog library will be used. This library is a tool that monitors a directory for file system events, such as the creation of a new file.

8.2. Deployment Workflow

The deployment system is designed as two distinct but connected components: an automated backend "Inference Service" that runs predictions and a frontend "Dashboard" that displays the results.

Component 1: The Automated Inference Service

This service is a persistent Python script running on the Ingest Server. Its sole job is to "watch" the data-in folder and automatically run the model on any new data.

Monitor Raw Data Folder: The watchdog library will be configured to monitor the 02_Data/raw/ directory, as defined in the Data Management Plan.

Trigger on New File: When a "Master" ESP32 node saves a new data file (e.g., 20251116T110400Z_SBM-AVR_ACU-01_T03_RAW.csv) in this directory, the watchdog script will detect this "file created" event.

Execute Prediction Pipeline: The detection event triggers a function that automates the entire processing-to-prediction pipeline: a. Load & Preprocess Data: The script loads the newly created raw CSV file. It then applies the appropriate preprocessing scripts (from 03_Code/2_preprocessing/) and calibration correction functions (CCFs) (from 01_Sensor_Calibration/) to clean and validate the data. b. Load Model: The script loads the best-performing, trained machine learning model (e.g., random_forest_final.pkl) from the 04_Outputs/models/ directory using the joblib or pickle library. c. Generate Prediction: The model's. predict() method is called on the newly processed sensor data. This will output a prediction (e.g., "Normal" or "Abnormal").

Store Prediction: The script opens a connection to the predictions.db SQLite database. It then inserts a new row into a table (e.g., predictions) containing the key information: the Unit ID, the timestamp, and the final prediction.

Component 2: The Streamlit Dashboard (app.py)

This is the interactive web application that the research team or maintenance staff will use. It is a single Python script (e.g., app.py) that is run from the server's terminal (e.g., streamlit run app.py).

Database Connection: When the user loads the web app, the Streamlit script connects to the predictions.db SQLite database to retrieve the latest data.

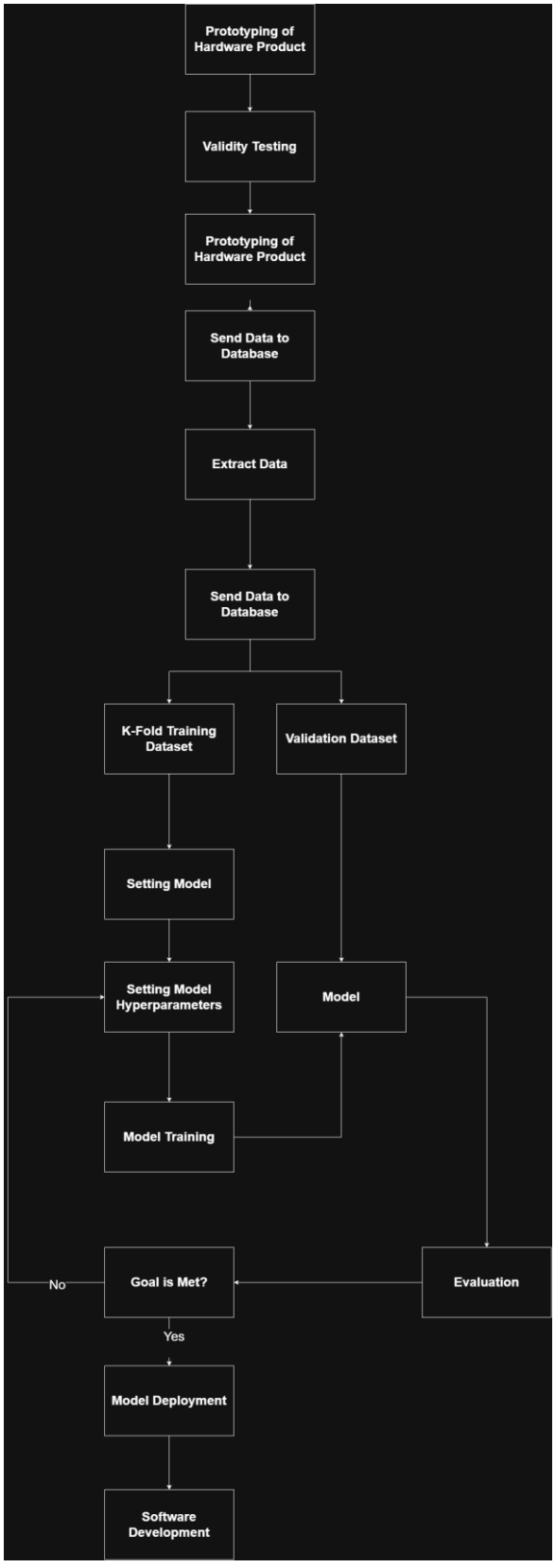
Main Status Dashboard: The default page will display a high-level overview of all 50 AC units. It will query the database to find the most recent prediction for each unit and display them on a simple table, color-coded for status (e.g., Green for "Normal," Red for "Abnormal").

Detailed Unit View: The dashboard will feature a dropdown menu (st.selectbox) allowing the user to select a specific AC Unit ID (e.g., COE_ACU-02).

Historical Analysis: Once a unit is selected, the app will re-query the database to fetch the entire prediction history for that single unit. This data will be displayed using Streamlit's built-in charting elements (e.g., st.line_chart or st.dataframe). This allows maintenance staff to not only see the current fault but also review the historical sensor data (like coil temperature, compressor current) that led up to the "Abnormal" prediction, providing critical context for diagnostics.

3.6 GENERAL PROJECT WORKFLOW

Generally, this project will follow a general workflow based on the previous chapters for the Product and Model Deployment as well as the Software Development. The following project workflow and projected gantt chart are shown below:

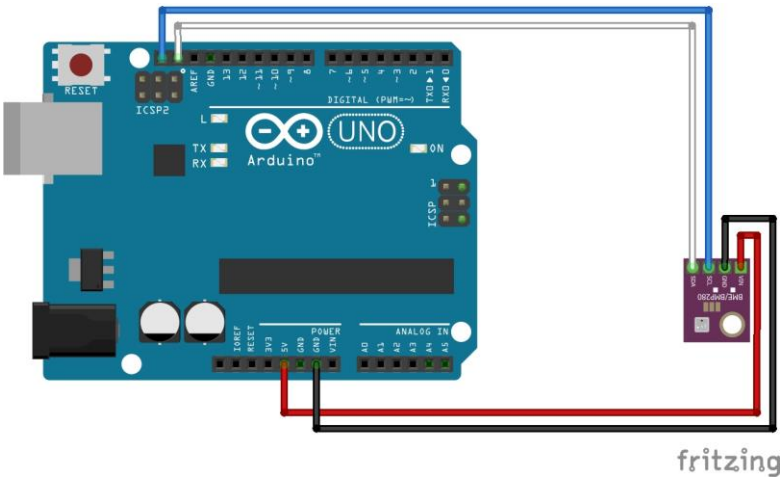

Prototyping of Hardware Product
Validity Testing
Prototyping of Hardware Product
Send Data to Database
Extract Data
Send Data to Database
K-Fold Training Dataset
Validation Dataset
Setting Model
Setting Model Hyperparameters
Model
Model Training
Evaluation
Goal is Met?
No
Yes
Model Deployment
Software Development

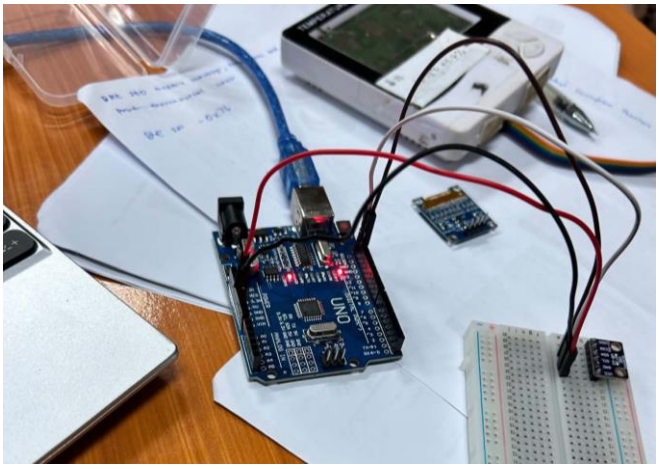
CHAPTER IV: RESULTS AND DISCUSSION

4.1 SENSORS VALIDITY EXPERIMENT

4.1.1 CALIBRATION OF BME 280

The validity experiment described in Chapter 3.1.6 was utilized to calibrate the temperature and humidity measurements of the BME280 sensor against a reference indoor thermometer. The pin configuration used during the experiment is presented below.





The BME280 validity experiment was conducted using 100 samples collected in a controlled environment, as presented in Appendix A.1. Linear regression analysis was performed in R using the collected data, producing the following results.

```
Input: R


model = lm(temp_data_and_humidty_dataa$`temp
device`~temp_data_and_humidty_dataa$`temp sensor`)
  summary(model)
```

```
Output: R


Call:
lm(formula = temp_data_and_humidty_dataa_1_$`temp device` ~
temp_data_and_humidty_dataa_1_$`temp sensor` -
  1)


Residuals:
    Min      1Q  Median      3Q     Max
-0.23014 -0.06543 -0.01838  0.04009  0.82283


Coefficients:
                                  Estimate Std. Error t value Pr(>|t|)
temp_data_and_humidty_dataa_1_$`temp sensor`   0.0005391    1909   <2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1
```

```
Residual standard error: 0.1329 on 99 degrees of freedom
Multiple R-squared:  1,  Adjusted R-squared:  1
F-statistic: 3.646e+06 on 1 and 99 DF,  p-value: < 2.2e-16
```

The results indicate a statistically significant calibration model ($p < 0.05$), demonstrating that the BME280 readings can be accurately adjusted to match those of the reference indoor thermometer. The analysis yielded the following calibration equation:

$$Ref = Sensor \times 1.0294412$$

The resulting calibration model indicates that the sensor readings should be multiplied by a slope factor of 1.0294412 to align with the measurements obtained from the reference instrument. By applying this calibration equation, the experimental code presented in Appendix A.1 can be modified to generate calibrated temperature readings.

Humidity calibration was performed using the same methodology. The corresponding data tables are provided in the Appendices. Based on the regression analysis, the following results were obtained.

Input: R

```
model = lm(temp_data_and_humidty_dataa_1_$`humidity 2 %`~temp_data_and_humidty_dataa_1_$`humidity 1 %`-1)
summary(model)
```

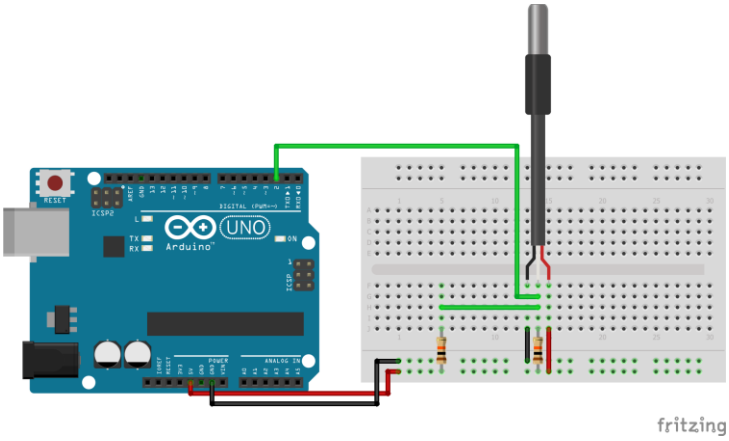
```
Output: R


Call:
lm(formula = temp_data_and_humidty_dataa_1_$`humidity 2 %` ~
  temp_data_and_humidty_dataa_1_$`humidity 1 %` - 1)


Residuals:
   Min      1Q  Median      3Q     Max
-1.03749 -0.13799  0.01012  0.16352  2.00433


Coefficients:
                                          Estimate Std. Error t value Pr(>|t|)
temp_data_and_humidty_dataa_1_$`humidity 1 %` 1.0579399  0.0008651    1223  <2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1


Residual standard error: 0.3348 on 99 degrees of freedom
Multiple R-squared:  0.9999,    Adjusted R-squared:  0.9999
F-statistic: 1.496e+06 on 1 and 99 DF,  p-value: < 2.2e-16
```

The humidity readings from the sensor can be calibrated using the following calibration equation, which was found to be statistically significant ($p < 0.05$):

$$Ref = Sensor \times 1.0579399$$

4.1.2 CALIBRATION OF DS18B20

The validity experiment described in Chapter 3.1.6 was also utilized to calibrate the temperature measurements of the DS18B20 sensors against a reference thermocouple. The calibration procedure followed the same methodology used for the BME280 sensor. The primary difference was that the DS18B20 sensors were evaluated under two controlled temperature conditions: normal and cold environments. The pin configuration used during the experiment is presented below.



The DS18B20 validity experiment was conducted using 100 samples from each temperature condition, as presented in Appendix A.4. The data collected from both the normal and cold trials were combined into a single dataset, and linear regression analysis was performed in R to develop calibration models for each probe.

Input: R

```
modelA = lm(Probe_A$Reference ~ Probe_A$Sensor)
summary(modelA)

modelB = lm(Probe_B$Reference ~ Probe_B$Sensor)
summary(modelB)

modelC = lm(Probe_C$Reference ~ Probe_C$Sensor)
summary(modelC)
```

Output: R

```
Call:
lm(formula = Probe_A$Reference ~ Probe_A$Sensor)

Residuals:
    Min      1Q  Median      3Q     Max
-9.9523 -0.0132  0.0698  0.1015  0.5758

Coefficients:
```

```
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  1.701471   0.188579   9.023   <2e-16 ***
Probe_A$Sensor 0.984997   0.008946 110.103   <2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 0.7182 on 198 degrees of freedom
Multiple R-squared:  0.9839,    Adjusted R-squared:  0.9838
F-statistic: 1.212e+04 on 1 and 198 DF,  p-value: < 2.2e-16


Call:
lm(formula = Probe_A$Reference ~ Probe_A$Sensor)

Residuals:
    Min      1Q  Median      3Q     Max
-9.9523 -0.0132  0.0698  0.1015  0.5758

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  1.701471   0.188579   9.023   <2e-16 ***
Probe_A$Sensor 0.984997   0.008946 110.103   <2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 0.7182 on 198 degrees of freedom
Multiple R-squared:  0.9839,    Adjusted R-squared:  0.9838
F-statistic: 1.212e+04 on 1 and 198 DF,  p-value: < 2.2e-16
```

```
Call:
lm(formula = Probe_B$Reference ~ Probe_B$Sensor)

Residuals:
    Min      1Q  Median      3Q     Max
-0.23984 -0.06551  0.01199  0.05003  0.19215

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)  1.8443655  0.0169324   108.9   <2e-16 ***
Probe_B$Sensor 0.9684665  0.0008711  1111.8   <2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 0.07977 on 198 degrees of freedom
Multiple R-squared:  0.9998,    Adjusted R-squared:  0.9998
F-statistic: 1.236e+06 on 1 and 198 DF,  p-value: < 2.2e-16
```

The results indicate that all regression models are statistically significant ($p < 0.05$). The resulting calibration equations were therefore used to adjust the Arduino sketch for each DS18B20 probe, enabling the sensors to produce measurements that more closely align with those of the reference thermocouple.

REFERENCES

[1] Tejani, A. (2024). AI-driven predictive maintenance in HVAC systems: Strategies for improving efficiency and reducing system downtime. ESP International Journal of Advancements in Science & Technology, 2(3), 6–19. https://doi.org/10.56472/25839233/IJAST-V2I3P102.

[2] Trivedi, S., Bhola, S., Archit Talegaonkar, Gaur, P., & Sharma, S. (2019). Predictive Maintenance of Air Conditioning Systems Using Supervised Machine Learning. https://doi.org/10.1109/isap48318.2019.9065995

[3] Singh, D., Arshad, M., Tyagi, B., & Kalia, G. (2023, October 12). Predictive Maintenance Strategies for HVAC Systems: Leveraging MPC, Dynamic Energy Performance Analysis, and ML Classification Models. https://www.researchgate.net/publication/374632248_Predictive_Maintenance_Strategies_for_HVAC_Systems_Leveraging_MPC_Dynamic_Energy_Performance_Analysis_and_ML_Classification_Models

[4] Sulaiman, N. A., Abdullah, M. P., Abdullah, H., Zainudin, M. N. S., & Md Yusop, A. (2020). Fault detection for air conditioning system using machine learning. *IAES International Journal of Artificial Intelligence (IJ-AI)*, *9*(1), 109. https://doi.org/10.11591/ijai.v9.i1.pp109-116

[5] Bouabdallaoui, Y., El Himer, S., & Ouladsine, M. (2021). Predictive Maintenance in Building Facilities: A Machine Learning Approach. Sensors, 21(18 56. https://doi.org/10.3390/s21186156

[6] Sharma, V., & Mistry, V. (2024). Machine learning algorithms for predictive maintenance in HVAC systems. *Zenodo*. https://doi.org/10.5281/zenodo.11079980

[7] Song, Y., Ma, Q., Zhang, T., Li, F., & Yu, Y. (2023). Research on Fault Diagnosis Strategy of Air-Conditioning Systems based on DPCA and Machine Learning. *Processes*, *11*(4), 1192. https://doi.org/10.3390/pr11041192

[8] Aji, A. S., Sashiomarda, J. A., & Handoko, D. (2020). Predictive maintenance magnetic sensor using random forest method. *Journal of Physics Conference Series*, *1528*(1), 012030. https://doi.org/10.1088/1742-6596/1528/1/012030

[9] Abood, A. M., Nasser, A. R., & Al-Khazraji, H. (2023). Predictive maintenance of electromechanical systems based on enhanced generative adversarial neural network with convolutional neural network. IAES International Journal of Artificial Intelligence, 12(4), 1704. https://doi.org/10.11591/ijai.v12.i4.pp1704-1712

[10] Sarker, I. H. (2021). Machine learning: algorithms, Real-World applications and research directions. *SN Computer Science*, *2*(3), 160. https://doi.org/10.1007/s42979-021-00592-x

[11] Hossain, E. (2023). *Machine learning crash course for engineers*. Springer. https://doi.org/10.1007/978-3-031-46989-3

[12] Raschka, S., Liu, Y., & Mirjalili, V. (2022). *Machine learning with PyTorch and scikit-learn*. Packt Publishing.

APPENDIX

A.1 BME 280 EXPERIMENTAL CODE

```
#include <Wire.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_BME280.h>

#define SEALEVELPRESSURE_HPA (1013.25)

Adafruit_BME280 bme;

void setup() {
  Serial.begin(115200);

  if (!bme.begin(0x76)) {
    Serial.println("Could not find a valid BME280 sensor, check wiring!");
    while (1);
  }
}

void loop() {

  Serial.print("Temperature = ");
  Serial.print(bme.readTemperature());
  Serial.println(" *C");

  Serial.print("Pressure = ");
  Serial.print(bme.readPressure() / 100.0F);
```

```
  Serial.println(" hPa");


  Serial.print("Approx. Altitude = ");
  Serial.print(bme.readAltitude(SEALEVELPRESSURE_HPA));
  Serial.println(" m");


  Serial.print("Humidity = ");
  Serial.print(bme.readHumidity());
  Serial.println(" %");


  Serial.println();
  delay(1000);
}
```

A.2 BME 280 VALIDITY EXPERIMENT TABLE

BME 280	Indoor Thermometer
24.66	25.6
24.69	25.6
24.66	25.6
24.86	25.6
24.71	25.6
24.71	25.6
24.75	25.6
24.76	25.5
24.83	25.5
24.79	25.5
24.75	25.5
24.61	25.5
24.58	25.5
24.64	25.5
24.64	25.5
24.66	25.5
24.55	25.5
24.57	25.5
24.64	25.5

24.65	25.5
24.64	25.5
24.58	25.5
24.64	25.5
24.66	25.4
24.65	25.4
24.65	25.4
24.69	25.4
24.72	25.4
24.71	25.4
24.67	25.4
24.69	25.4
24.69	25.4
24.7	25.4
24.72	25.4
24.71	25.4
24.72	25.4
24.75	25.4
24.79	25.4
24.81	25.4
24.77	25.4
24.75	25.4
24.77	25.4
24.79	25.4
24.8	25.4

24.74	25.4
24.7	25.4
24.73	25.4
24.72	25.4
24.7	25.4
24.66	25.4
24.7	25.4
24.73	25.4
24.69	25.4
24.72	25.4
24.73	25.4
24.7	25.4
24.71	25.4
24.68	25.4
24.69	25.4
24.66	25.4
24.61	25.4
24.61	25.4
24.56	25.4
24.59	25.3
24.61	25.3
24.63	25.3
24.64	25.3
24.65	25.3
24.66	25.3

24.64	25.3
24.64	25.3
24.59	25.3
24.57	25.3
24.58	25.3
24.59	25.3
24.63	25.3
24.59	25.3
24.6	25.3
24.8	25.3
24.58	25.3
24.53	25.3
24.54	25.3
24.54	25.3
24.5	25.3
24.51	25.3
24.54	25.3
24.63	25.3
24.65	25.2
24.69	25.2
24.64	25.2
24.62	25.2
24.58	25.2
24.61	25.2
24.62	25.2

24.6	25.2
24.62	25.3
24.62	25.2
23.68	25.2
24.66	25.2
24.68	25.2

BME 280	Indoor Humidity
38.35	41
38.9	41
38.75	41
38.81	41
38.83	41
38.72	41
38.94	41
38.68	41
38.56	41
38.45	41
38.56	41
38.75	41
38.84	41
38.84	41
38.91	41
38.39	41

39	41
39.13	41
39.16	41
38.72	41
39.06	41
38.95	41
39.02	41
38.91	41
38.8	41
38.74	41
38.88	41
38.81	41
38.61	41
38.62	41
38.69	41
38.78	41
39.05	41
38.75	41
38.95	41
38.99	41
39.13	41
38.54	41
38.4	41
38.55	41
38.56	41

38.69	41
38.57	41
38.44	41
38.53	41
38.7	41
38.77	41
38.79	41
38.79	41
38.49	41
38.6	41
38.6	41
38.61	41
38.6	41
38.65	41
38.5	41
38.62	41
38.54	41
39.04	41
38.76	41
38.67	41
38.67	41
38.71	41
38.76	41
39	41
38.9	41

38.39	41
38.79	41
38.73	41
38.84	41
38.56	41
38.56	41
38.7	41
38.67	41
38.76	41
38.6	41
38.77	41
38.62	41
38.62	41
36.86	41
38.76	41
38.66	41
38.71	41
38.68	41
38.81	41
38.98	41
38.8	41
38.92	41
38.56	41
38.56	41
38.56	41

38.79	41
38.95	41
38.76	41
38.55	41
38.79	40
38.79	40
38.32	40
38.65	40
38.42	40

A.3 BME 280 VALIDITY CALIBRATED CODE

```
#include <Wire.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_BME280.h>

#define SEALEVELPRESSURE_HPA (1013.25)

Adafruit_BME280 bme;

void setup() {
  Serial.begin(115200);

  if (!bme.begin(0x76)) {
    Serial.println("Could not find a valid BME280 sensor, check wiring!");
    while (1);
  }
}

void loop() {
  // Raw sensor
  float sensorTemp = bme.readTemperature();
  float sensorHumid = bme.readHumidity();

  // Calibration
  float calTemp= sensorTemp * 1.0294412;
  float calHum = sensorHumid * 1.0579399;
```

```
  Serial.print("Temperature = ");
  Serial.print(calTemp);
  Serial.println(" *C");

  Serial.print("Pressure = ");
  Serial.print(bme.readPressure() / 100.0F);
  Serial.println(" hPa");

  Serial.print("Approx. Altitude = ");
  Serial.print(bme.readAltitude(SEALEVELPRESSURE_HPA));
  Serial.println(" m");

  Serial.print("Humidity = ");
  Serial.print(calHum);
  Serial.println(" %");

  Serial.println();
  delay(1000);
}
```

A.4 DS18B20 PROBE A TABLE

DS18B20 PROBE A	Reference
26.37	27.8

26.44	27.7
26.37	27.8
26.37	27.7
26.37	27.7
26.37	27.7
26.37	27.7
26.37	27.6
26.31	27.6
26.31	27.7
26.31	27.7
26.31	27.7
26.31	27.7
26.25	27.7
26.25	27.7
26.19	27.6
26.19	27
26.19	27.6
26.19	27.6
26.19	27.6
26.19	27.6
26.19	27.6
26.19	27.6
26.19	27.6
26.19	27.6
26.19	27.6

26.12	27.6
26.12	27.5
26.12	27.5
26.06	27.6
26.12	27.5
26.06	27.5
26.12	27.5
26.06	27.5
26.06	27.5
26.06	27.5
26	27.4
26	27.4
26.06	27.4
26	27.4
26	27.4
26	27.4
26	27.4
26	27.4
26	27.3
25.94	27.3
25.94	27.3
25.94	17.3
25.94	27.3
25.94	27.3
25.94	27.3

25.87	27.3
25.94	27.3
26	27.3
25.87	27.3
25.87	27.3
25.87	27.3
25.94	27.2
25.87	27.2
25.87	27.3
25.87	27.3
25.81	27.3
25.81	27.3
25.81	27.3
25.87	27.3
25.81	27.3
25.81	27.3
25.81	27.2
25.81	27.2
25.81	27.2
25.81	27.2
25.75	27.2
25.81	27.3
25.81	27.2
25.75	27.2
25.81	27.7

25.81	27.3
25.81	27.2
25.75	27.3
25.81	27.2
25.75	27.2
25.69	27.2
25.75	27.2
25.69	27.2
25.75	27.2
25.69	27.2
25.69	27.2
25.69	27.2
25.69	27.2
25.69	27.2
25.75	27.1
25.69	27.1
25.69	27.1
25.69	27.1
25.69	27.1
25.69	27.1
25.69	27.1
25.69	27.1
25.69	27.1
25.62	27.1
25.6	27.1
14	15.4

14	15.4
14	15.4
14	15.4
14	15.4
14.06	15.4
14.06	15.5
14.06	15.5
14.06	15.5
14.06	15.6
14.06	15.6
14.13	15.6
14.13	15.6
14.13	15.6
14.19	15.6
14.19	15.5
14.19	15.6
14.25	15.7
14.25	15.7
14.25	15.7
14.25	15.7
14.25	15.7
14.25	15.7
14.31	15.7
14.31	15.7
14.31	15.7

14.31	15.8
14.31	15.8
14.38	15.8
14.38	15.8
14.38	15.8
14.38	15.8
14.44	15.8
14.44	15.8
14.44	15.8
14.5	15.8
14.44	15.9
14.44	15.9
14.56	15.9
14.5	16
14.5	16.1
14.5	16
14.56	16.1
14.56	16.1
14.56	16.1
14.56	16.1
14.63	16.1
14.63	16.2
14.63	16.2
14.63	16.2
14.63	16.2

14.69	16.2
14.69	16.2
14.69	16.2
14.75	16.2
14.69	16.2
14.75	16.2
14.75	16.3
14.75	16.3
14.75	16.3
14.75	16.3
14.75	16.3
14.81	16.3
14.81	16.3
14.88	16.3
14.81	16.3
14.88	16.3
14.81	16.4
14.88	16.4
14.94	16.4
14.94	16.4
14.88	16.4
14.94	16.5
14.88	16.5
15	16.5
14.94	16.6

14.94	16.6
15	16.6
14.94	16.6
15	16.6
15	16.6
15.06	16.6
15.06	16.6
15.06	16.6
15.06	16.6
15.06	16.6
15.13	16.7
15.13	16.7
15.13	16.7
15.13	16.7
15.13	16.7
15.19	16.7
15.19	16.7
15.19	16.7
15.19	16.7
15.25	16.7
15.19	16.7
15.25	16.7
15.25	16
15.25	16.8

A.5 DS18B20 PROBE B TABLE

DS18B20 PROBE B	Reference
25	26.2
25	26.1
25.06	26

25	26.1
25.06	26.1
25	26.1
25	26.1
25	26.1
25	26
24.94	26.1
25	26.1
25	26.1
25	26.1
25	26.1
24.94	26.1
24.94	26.1
24.94	26
24.94	26
24.94	26
24.94	26
24.94	25.9
24.94	26
25	26
24.87	25.9
24.94	25.9
24.94	25.9
24.87	25.9
24.87	26

24.94	25.9
24.94	25.9
24.94	25.9
24.94	25.9
24.87	25.9
24.87	25.9
24.87	25.9
24.87	25.9
24.94	25.9
24.87	25.9
24.87	25.9
24.87	25.9
24.81	25.9
24.81	25.9
24.81	25.9
24.81	25.9
24.81	25.8
24.81	25.9
24.87	25.8
24.87	25.8
24.81	25.8
24.81	25.8
24.81	25.8
24.81	25.8
24.81	25.8

24.81	25.8
24.81	25.8
24.81	25.8
24.81	25.8
24.75	25.8
24.81	25.8
24.81	25.8
24.75	25.8
24.75	25.8
24.75	25.8
24.75	25.8
24.69	25.8
24.69	25.8
24.69	25.8
24.69	25.8
24.69	25.8
24.69	25.8
24.69	25.8
24.69	25.8
24.69	25.8
24.69	25.8
24.69	25.7
24.62	25.8
24.62	25.7
24.62	25.8
24.69	25.8

24.62	25.7
24.62	25.7
24.69	25.7
24.62	25.7
24.62	25.7
24.62	25.7
24.62	25.7
24.62	25.7
24.62	25.7
24.62	25.7
24.56	25.7
24.62	25.7
24.62	25.7
24.62	25.7
24.62	25.7
24.56	25.7
24.56	25.7
24.56	25.7
24.56	25.7
24.56	25.7
24.56	25.7
24.56	25.7
24.56	25.7
11.63	12.9
11.56	12.9
11.56	12.9

11.56	12.9
11.5	12.8
11.5	12.8
11.44	12.8
11.44	12.8
11.44	12.8
11.38	12.8
11.38	12.8
11.38	12.8
11.38	12.9
11.38	12.9
11.38	12.8
11.38	12.9
11.44	12.9
11.44	12.9
11.44	12.9
11.38	12.9
11.44	12.9
11.44	12.9
11.44	12.9
11.44	12.9
11.44	12.9
11.5	12.9
11.5	12.9
11.5	12.9
11.5	12.9

11.5	12.9
11.5	12.9
11.56	12.9
11.56	12.9
11.56	12.9
11.56	12.9
11.56	12.8
11.56	12.9
11.56	12.9
11.56	12.9
11.56	13
11.56	13
11.63	13
11.63	13.1
11.69	13.1
11.63	13.1
11.69	13.2
11.69	13.2
11.75	13.2
11.75	13.3
11.75	13.3
11.75	13.3
11.75	13.3
11.81	13.3
11.81	13.4

11.81	13.4
11.81	13.4
11.81	13.4
11.88	13.4
11.88	13.5
11.88	13.5
11.94	13.5
12	13.6
11.94	13.6
11.94	13.5
12.06	13.5
12	13.6
12.06	13.6
12.06	13.6
12.06	13.7
12.13	13.6
12.13	13.6
12.13	13.6
12.13	13.7
12.19	13.7
12.19	13.7
12.19	13.7
12.25	13.7
12.19	13.7
12.19	13.7

12.25	13.8
12.31	13.8
12.31	13.8
12.31	13.8
12.31	13.8
12.38	13.8
12.31	13.9
12.38	13.9
12.38	13.9
12.38	13.9
12.44	13.9
12.44	13.9
12.44	13.9
12.44	14
12.5	14
12.5	14
12.44	14
12.56	14.1
12.56	14.1
12.56	14.1
12.56	14.1
12.63	14.1

A.6 DS18B20 PROBE C TABLE

DS18B20 PROBE B	Reference
25	26.2
25	26.1
25.06	26
25	26.1
25.06	26.1
25	26.1
25	26.1
25	26.1
25	26
24.94	26.1
25	26.1
25	26.1
25	26.1
25	26.1
24.94	26.1
24.94	26.1
24.94	26
24.94	26
24.94	26
24.94	26
24.94	25.9

24.94	26
25	26
24.87	25.9
24.94	25.9
24.94	25.9
24.87	25.9
24.87	26
24.94	25.9
24.94	25.9
24.94	25.9
24.94	25.9
24.87	25.9
24.87	25.9
24.87	25.9
24.87	25.9
24.94	25.9
24.87	25.9
24.87	25.9
24.87	25.9
24.87	25.9
24.81	25.9
24.81	25.9
24.81	25.9
24.81	25.9
24.81	25.8
24.81	25.9

24.87	25.8
24.87	25.8
24.81	25.8
24.81	25.8
24.81	25.8
24.81	25.8
24.81	25.8
24.81	25.8
24.81	25.8
24.81	25.8
24.81	25.8
24.81	25.8
24.75	25.8
24.81	25.8
24.81	25.8
24.75	25.8
24.75	25.8
24.75	25.8
24.75	25.8
24.69	25.8
24.69	25.8
24.69	25.8
24.69	25.8
24.69	25.8
24.69	25.8
24.69	25.8

24.69	25.8
24.69	25.8
24.69	25.7
24.62	25.8
24.62	25.7
24.62	25.8
24.69	25.8
24.62	25.7
24.62	25.7
24.69	25.7
24.62	25.7
24.62	25.7
24.62	25.7
24.62	25.7
24.62	25.7
24.62	25.7
24.62	25.7
24.62	25.7
24.56	25.7
24.62	25.7
24.62	25.7
24.62	25.7
24.62	25.7
24.56	25.7
24.56	25.7
24.56	25.7

24.56	25.7
24.56	25.7
24.56	25.7
24.56	25.7
11.63	12.9
11.56	12.9
11.56	12.9
11.56	12.9
11.5	12.8
11.5	12.8
11.44	12.8
11.44	12.8
11.44	12.8
11.38	12.8
11.38	12.8
11.38	12.8
11.38	12.9
11.38	12.9
11.38	12.8
11.38	12.9
11.44	12.9
11.44	12.9
11.44	12.9
11.38	12.9
11.44	12.9

11.44	12.9
11.44	12.9
11.44	12.9
11.5	12.9
11.5	12.9
11.5	12.9
11.5	12.9
11.5	12.9
11.5	12.9
11.56	12.9
11.56	12.9
11.56	12.9
11.56	12.9
11.56	12.8
11.56	12.9
11.56	12.9
11.56	12.9
11.56	13
11.56	13
11.63	13
11.63	13.1
11.69	13.1
11.63	13.1
11.69	13.2
11.69	13.2

11.75	13.2
11.75	13.3
11.75	13.3
11.75	13.3
11.75	13.3
11.81	13.3
11.81	13.4
11.81	13.4
11.81	13.4
11.81	13.4
11.88	13.4
11.88	13.5
11.88	13.5
11.94	13.5
12	13.6
11.94	13.6
11.94	13.5
12.06	13.5
12	13.6
12.06	13.6
12.06	13.6
12.06	13.7
12.13	13.6
12.13	13.6
12.13	13.6

12.13	13.7
12.19	13.7
12.19	13.7
12.19	13.7
12.25	13.7
12.19	13.7
12.19	13.7
12.25	13.8
12.31	13.8
12.31	13.8
12.31	13.8
12.31	13.8
12.38	13.8
12.31	13.9
12.38	13.9
12.38	13.9
12.38	13.9
12.44	13.9
12.44	13.9
12.44	13.9
12.44	14
12.5	14
12.5	14
12.44	14
12.56	14.1

12.56	14.1
12.56	14.1
12.56	14.1
12.63	14.1

A.7 DS18B20 EXAMPLE EXPERIMENTAL CODE

```
#include <OneWire.h>
#include <DallasTemperature.h>

// Data wire connected to digital pin 10
#define ONE_WIRE_BUS 10

// Setup OneWire instance
OneWire oneWire(ONE_WIRE_BUS);

// Pass OneWire reference to DallasTemperature library
DallasTemperature sensors(&oneWire);

void setup(void) {
 Serial.begin(115200);
 sensors.begin();
}

void loop(void) {

 // Request temperature reading
 sensors.requestTemperatures();

 // Raw sensor temperature
 float rawTemp = sensors.getTempCByIndex(0);

 // Calibration equation from regression output
```

```
 float calibratedTemp = 1.701471 + (0.984997 * rawTemp);


 // Print calibrated temperature
 Serial.print("Calibrated Temperature: ");
 Serial.print(calibratedTemp);
 Serial.println(" °C");


 delay(100);
}
```

CURRICULUM VITAE

SIMON FRANCE SULIBIO

Brgy. Crossing, Libona Purok 3B, Bukidnon

(0953) 168 9647

20230029109@my.xu.edu.ph

PERSONAL INFORMATION

Age: 21 years old

Sex: Male

Birthdate: January 21,2004

Birthplace: Crossing, Libona, Bukidnon

Religion: Roman Catholic

EDUCATIONAL BACKGROUND

Tertiary Education

School: Xavier University - Ateneo de Cagayan

Address: Corrales Avenue, Cagayan de Oro City, 9000, Philippines

School Year: College of Engineering (2023 - Present)

Senior High and Secondary Education

Strand: Science, Technology, Engineering and Mathematics strand (STEM)

School: Libona National High School

Address: Crossing, Libona, Bukidnon

School Year: 2021 – 2023 (Senior High school), 2017- 2021 (High School)

Primary Education

School: Crossing Elementary Central School

Address; Crossing, Libona, Bukidnon

School Year: 2011-2017

SKILLS:

- Skilled in arts
- Stress Management Maintains focus and productivity in fast-paced or demanding environment
- Strong work ethics

CURRICULUM VITAE

COLLIN BRANDON O. ASIO

Brgy. Macasandig, Cagayan De Oro City, 9000, Philippines

(0991) 647 0790

200720209@my.xu.edu.ph

PERSONAL BACKGROUND

Age: 21 years old

Birthplace: Cagayan de Oro City

Sex: Male

Religion: Roman Catholic

Birthdate: July 6, 2004

EDUCATIONAL BACKGROUND

Tertiary Education:

School: Xavier University - Ateneo de Cagayan

Address: Corrales Avenue, Cagayan de Oro City, 9000, Philippines

School Year: College of Engineering (2023 - Present)

Senior High and Secondary Education:

School: Xavier University Ateneo de Cagayan

Address: Masterson Avenue, Cagayan de Oro City,9000, Philippines

School Year: 2021 – 2023 (Senior High), 2017- 2021 (Junior High)

Primary Education:

School: Xavier University – Ateneo de Cagayan

Address: Macasandig, Tomas Saco St. Cagayan de Oro City, 9000, Philippines

School Year: 2007- 2017

SKILLS:

- Team player
- Has Background in programming
- Creative in AutoCAD
- Can work under pressure, Organize and has Strong work ethics

CURRICULUM VITAE

JOHN RONALD HOWELL B. PACALDO

Lawesbra, Lapasan, Cagayan de Oro City

(0945) 209 9154

200931726@my.xu.edu.ph

PERSONAL BACKGROUND

Age: 20 years old

Birthplace: Cagayan de Oro City

Sex: Male

Religion: Roman Catholic

Birthdate: October 11, 2005

EDUCATIONAL BACKGROUND

Tertiary Education:

School: Xavier University - Ateneo de Cagayan

Address: Corrales Avenue, Cagayan de Oro City, 9000, Philippines

School Year: College of Engineering (2023 - Present)

Senior high and Secondary Education:

Strand: Science, Technology, Engineering and Mathematics strand (STEM)

Senior High School: Xavier University Ateneo de Cagayan Senior High

Address: Masterson Avenue, Cagayan de Oro City,9000, Philippines

Junior High: Misamis Oriental General Comprehensive High School

Address: FJMW+5HW, Don Apolinar Velez St, Cagayan De Oro City

School Year: School Year: 2021 – 2023 (Senior High), 2017- 2021 (Junior High)

Primary Education:

School: Xavier University – Ateneo de Cagayan

Address: Macasandig, Tomas Saco St. Cagayan de Oro City, 9000, Philippines

School Year: 2007- 2017

SKILLS

- **Programming, AutoCAD and Fusion 360**
- **Team Player**